软件工程

第2讲　软件项目管理

本章内容

- 项目管理
- 项目度量
- 项目估算
- 项目计划
- 风险管理
- 质量管理



第2讲　软件项目管理

2.1 项目管理的概念

项目定义

项目就是在既定的资源和要求的限制下，为实现某种目标而相互联系的一次性的工作任务。

中国项目管理研究委员会对项目的定义：项目是一个特殊的将被完成的有限任务；它是在一定时间内，满足一系列特定目标的多项相关工作的总称。

项目的基本特征

① **明确的目标**
- 项目的目标可能是产品，也可能是服务。
- 项目有最终可交付的成果。

② **项目独特性**
- 项目所涉及的某些内容是以前没有做过的。
- 具有重复的要素并不能够改变其整体的唯一性。

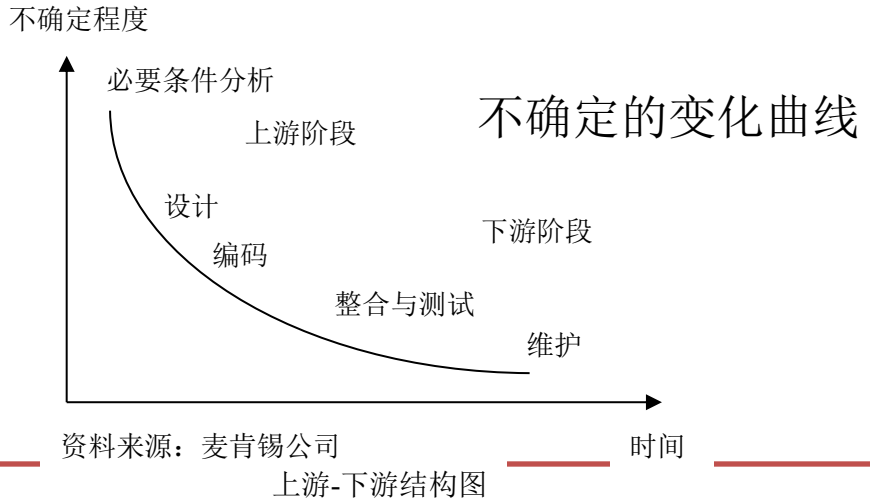
③ **项目的时限性**
- 时限性指每个项目都具有明确的开始和结束时间与标志，项目不能重复实施。

④ **项目的不确定性**
- 有时很难确切定义项目的目标或准确地估算出所需要的时间和成本，影响项目目标的实现。

⑤ **结果的不可逆转性**

软件项目的特点

1. 目标的渐进性
2. 项目的阶段性
3. 智力密集型
4. 不确定性



不确定的变化曲线

资料来源：麦肯锡公司

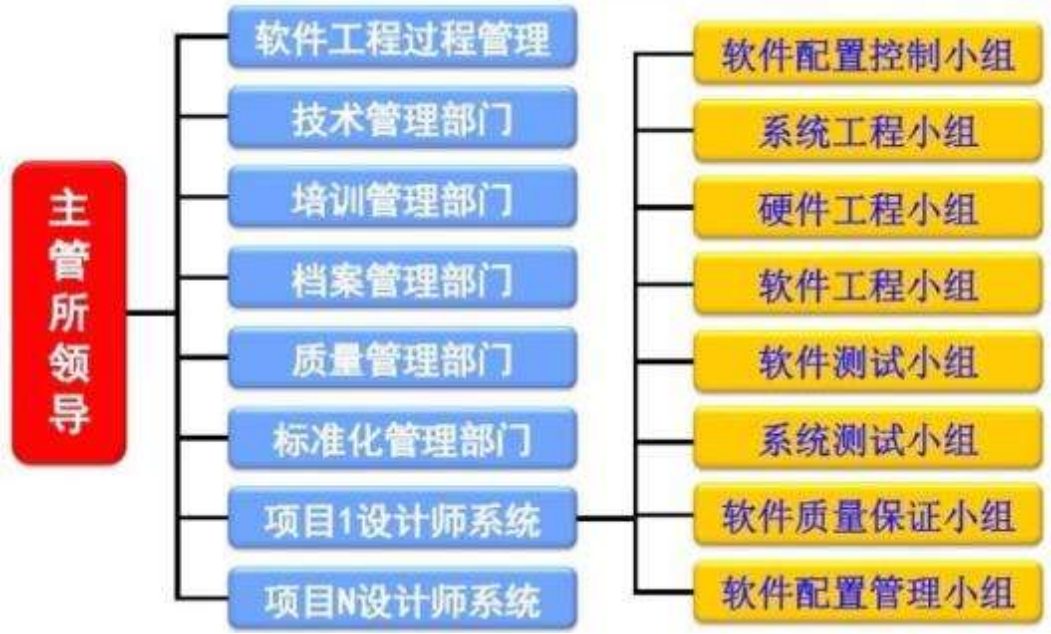
上游-下游结构图

软件项目更多的不确定性

- **不清晰的客户需求。**
  - 在软件业，客户的需要很难确定。
  - 结构常常必须保持开放，以结合接下来的变化。
- **设计不完全可以预测。**
  - 有时，对于如何构造软件所做的决定后来被证明是错的。
- **不断变化的需求。**
  - 在一开始正确表述了的需求常常在后来发生变化。
- **不断变化的技术。**
  - 软件系统，特别是软件产品，高度依赖于它们工作的技术环境。

软件项目的成功原则

- 平衡原则：需求、资源、工期、质量四个要素之间的平衡关系
- 高效原则：人员、目标、沟通、激励
- 分解原则：化繁为简，各个击破
- 实时控制原则
- 分类管理原则
- 简单有效原则
- 规模控制原则



管理

- 所谓管理就是通过计划、组织和控制等一系列活动，合理地配置和使用各种资源，以达到既定目标的过程。
- 软件项目管理先于任何技术活动之前开始，并且贯穿于软件的整个生命周期之中。

项目的管理过程

- 项目启动
- 项目计划
- 项目执行与控制
- 项目收尾

管理的范围(3P):

- 人员（People）
- 问题（Problem）
- 过程（Process）
- 项目（Project）

管理活动

- 提出书面建议
- 项目规划和进度
- 项目成本
- 项目监督和评审
- 人员选择和评价
- 写作并陈述工作报告

项目管理的九大知识领域

项目管理的九大知识领域是指作为项目经理必须具备与掌握的九大块重要知识与能力：

- 项目整体管理知识
- 项目范围管理知识
- 项目的时间管理知识
- 项目的成本管理知识
- 项目人力资源管理知识
- 项目的质量管理知识
- 项目的沟通管理知识
- 项目的风险管理知识
- 项目的采购管理知识

核心的四大知识领域是：

- 范围
- 时间
- 成本
- 质量管理

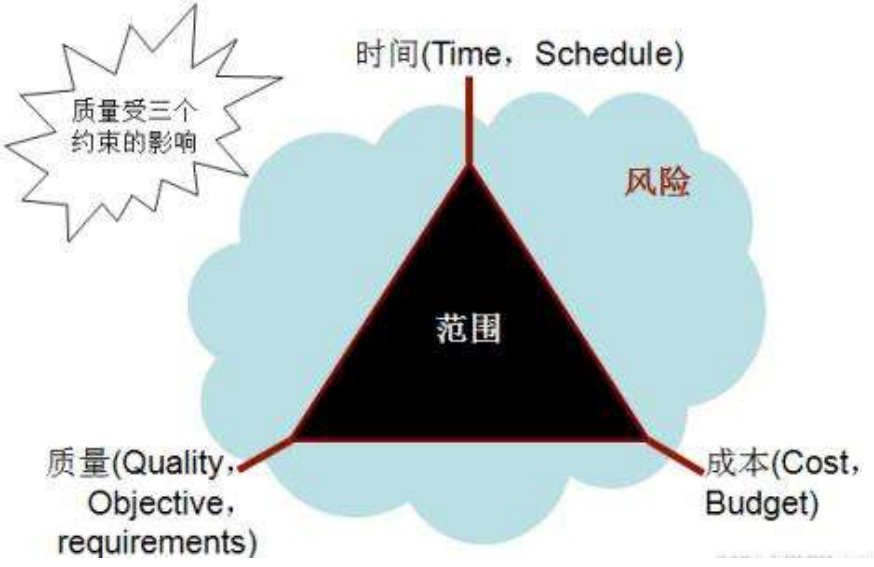


项目范围管理

- 项目范围管理是对项目应该包括什么和不应该包括什么进行相应的定义和控制。它包括用以保证项目能按要求的范围完成所涉及的所有过程，包括：
  - 确定项目的需求
  - 定义规划项目的范围
  - 范围管理的实施
  - 范围的变更控制管理
  - 范围核实

- 项目的“三约束条件”：
  - 范围
  - 时间
  - 成本





项目整体管理

- 项目整体管理，一方面体现完整和善始善终的意思，所以整体管理是唯一贯穿启动到收尾所有过程组的知识体系，从项目启动到项目收尾项目整体管理都得管。
- 最小项目管理活动集：
  ① 启动-制定项目单程并确定初步范围
  ② 计划-制定项目管理计划
  ③ 执行-制导和管理项目的执行活动
  ④ 监控-监控项目执行并进行整体变量控制
  ⑤ 收尾-项目收尾和结束

项目管理的7个过程：

① 制定项目章程：制定一份正式批准项目或阶段的文件，并记录能反映干系人需要和期望的初步要求的过程；

② 制定项目范围说明书（初步）

③ 制定项目管理计划：对定义、编制、整合和协调所有子计划所必需的行动进行记录的过程；

④ 指导和管理项目执行：为实现项目目标而执行项目管理计划中所确定的工作的过程；

⑤ 监督和控制项目工作：跟踪、审查和调整项目进展，以实现项目管理计划中确定的绩效目标的过程；

⑥ 整体变更控制：审查所有变更请求，批准变更，管理对可交付成果、组织过程资产、项目文件和项目管理计划的变更的过程；

⑦ 项目收尾：完结所有项目管理过程组的所有活动，以正式结束项目或阶段的过程。

软件项目管理

- 软件项目管理就是为了使软件项目能够按照预定的成本、进度、质量顺利完成，而对人员（People）、产品（Product）、过程（Process）和项目（Project）进行分析和管理的活动。
- 软件项目管理的对象是软件工程项目。它所涉及的范围覆盖了整个软件工程过程。
- 软件项目管理的主要方面是软件项目的工作范围、可能风险、需要资源（人、硬件/软件）、要实现的任务、经历的里程碑、花费工作量（成本）、进度安排等。

第2讲　软件项目管理

2.2 项目策划

计划的类型

- 开发计划
- 质量计划
- 验收计划
- 配置管理计划

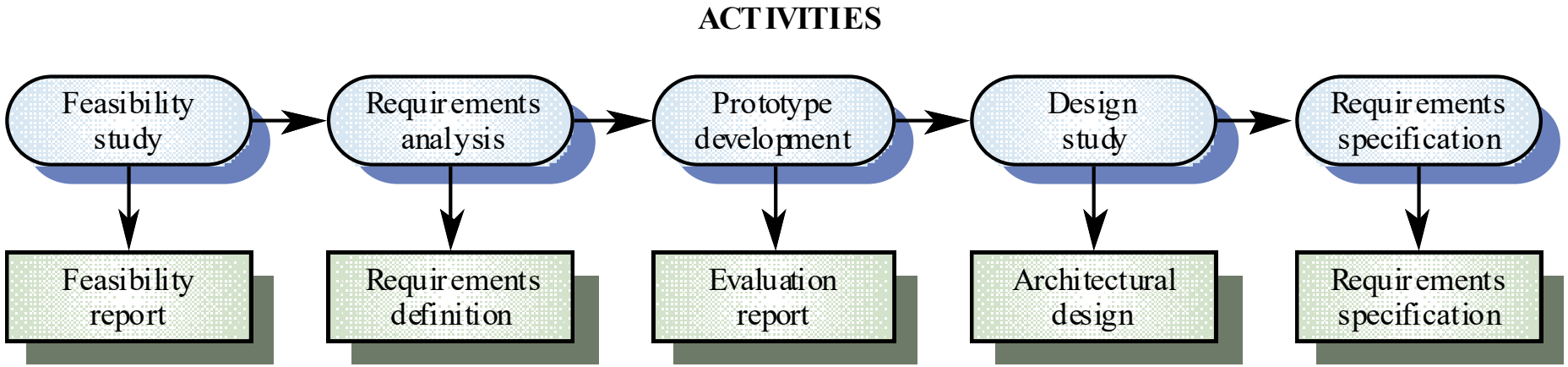
项目计划

① 引言
② 项目组织
③ 风险分析
④ 硬件和软件资源需求
⑤ 工作分解
⑥ 项目进度
⑦ 监控和报告机制

业内共识

没有检查点，项目难进展；
不设里程碑，项目往后推。

例：需求过程中的里程碑



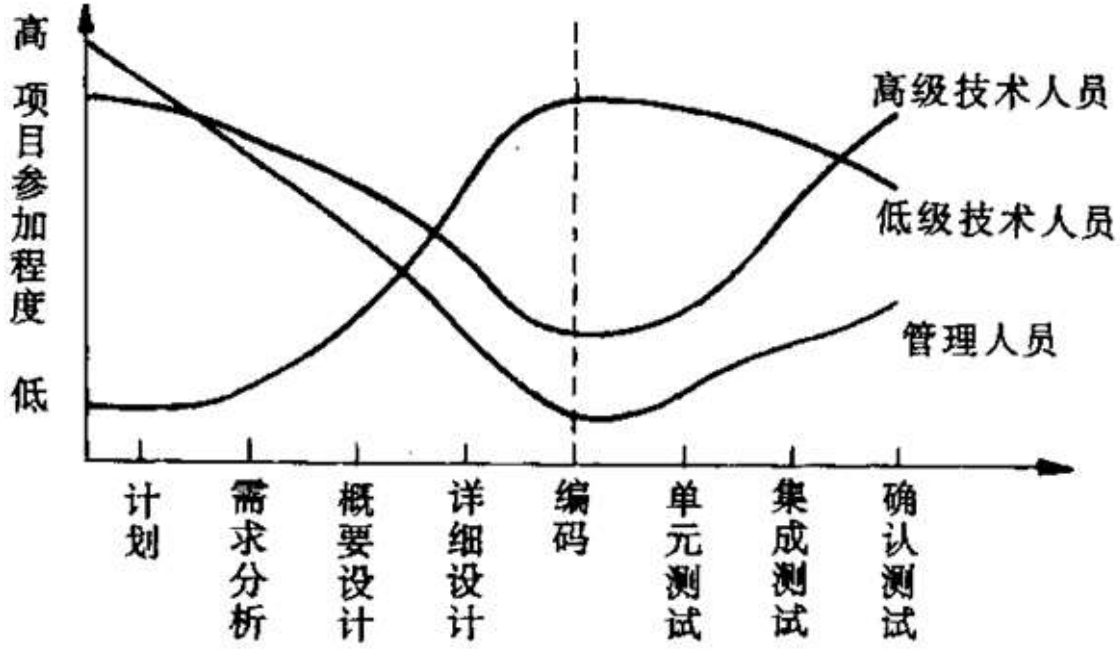


图　人员参加程度曲线图

项目计划的主要内容

软件项目项目范围

- 项目目标
- 主要功能
- 性能限制
- 系统接口
- 特殊要求（非功能需求）
- 开发概述

资源

- 人力资源
- 硬件资源
- 软件资源
- 知识资产
- 工具包

进度安排的主要的方法：

- 工程网络图
- Gantt图
- 任务资源表



估算

- **规模估算**
- **成本估算**

度量

- **规模度**
- **质量度量**
- **功能度量**

第2讲　软件项目管理

2.3 软件规模估算

课堂讨论

- 五个程序段的功能各是什么？
- 对这五个程序你有什么评论？

```
return a + b;
```



```
return a - (-b);
```



```
int k = b > 0 ? 1 : -1;
for (int i = 0; i != b; i+=k)
{
    a += k;
}
return a;
```



```
int i = Integer.MIN_VALUE);
while (i++ < Integer.MAX_VALUE)
{
    if (i - a == b && i - b == a)
    {
        return i;
    }
}
return 0;
```



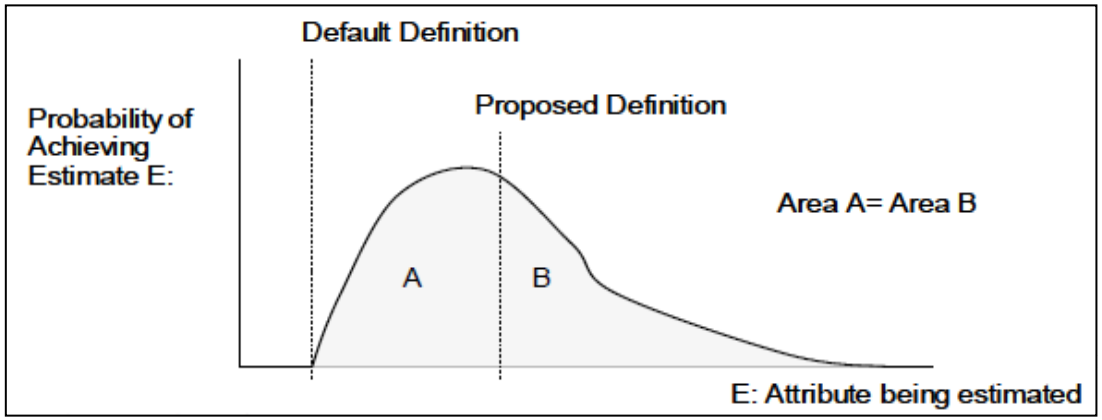
```
int n = 0;
boolean c = false;
for (int i = 1; i != 0; i = i << 1)
{
    boolean _a = (a & 1) == 1;
    boolean _b = (b & 1) == 1;
    boolean _n = (_a ^ _b) ^ c;
    c = _a & (_b | c) | (_b & c);
    n = n | (_n ? i : 0);
    a = a >> 1;
    b = b >> 1;
}
return n;
```



2.3.1 什么是估算

估算＝估计＋计划

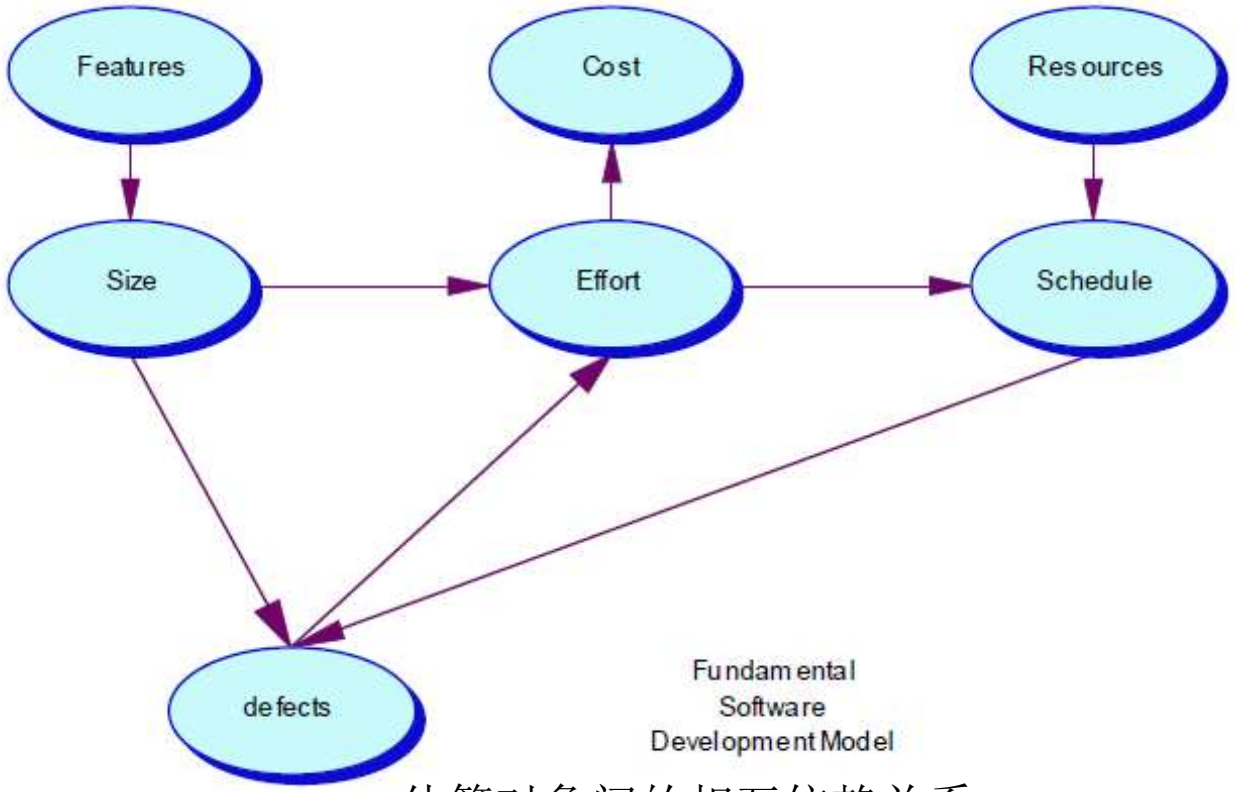
- 估算的作用
  - 软件估算一直是一项重要而艰巨的任务，准确的估计是进行有效的项目计划、跟踪和控制的基础。
  - 确定时间、人时及项目成本
  - 对活动、工作成果以及各种角色（包括用户）建立适当的预期
  - 确定项目里程碑
  - 估算以下各项的影响
    - 增加需求
    - 改变资源
    - 用项目监控因素来影响产品的质量



一个真实的估算，是一个预期结果及其允许偏差的范围，实际结果最大概率地落在这个范围内。

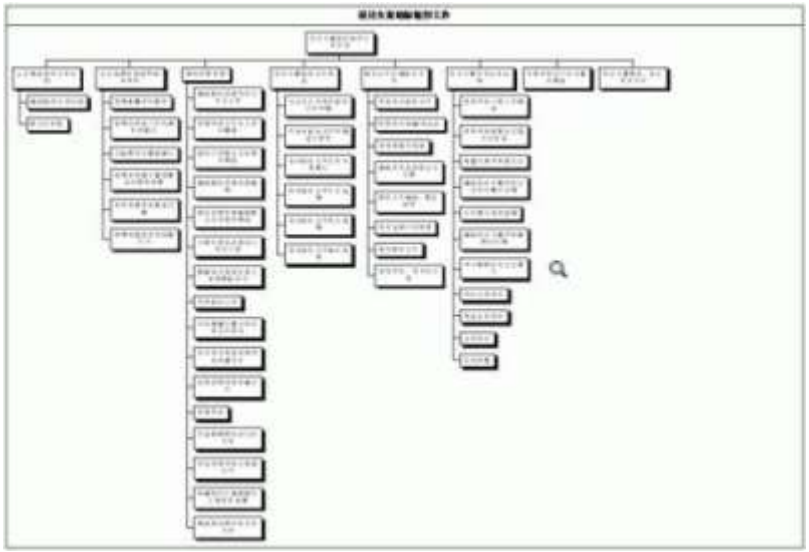
主要估算对象

- 规模（Size，all major work products and activities）
- 关键资源（Critical Project Resources）
- 工作量（Effort）
- 进度（Schedule）
- 成本（Cost）
- 风险（Risk）



估算对象间的相互依赖关系

工作任务分解法（WBS）



- 工作分解结构（Work Breakdown Structure，简称WBS）：以可交付成果为导向对项目要素进行的分组，它归纳和定义了项目的整个工作范围每下降一层代表对项目工作的更详细定义。
- WBS同时也是控制项目变更的重要基础。项目范围是由WBS定义的，所以WBS也是一个项目的综合工具。

WBS把一个项目，按一定的原则分解，即：

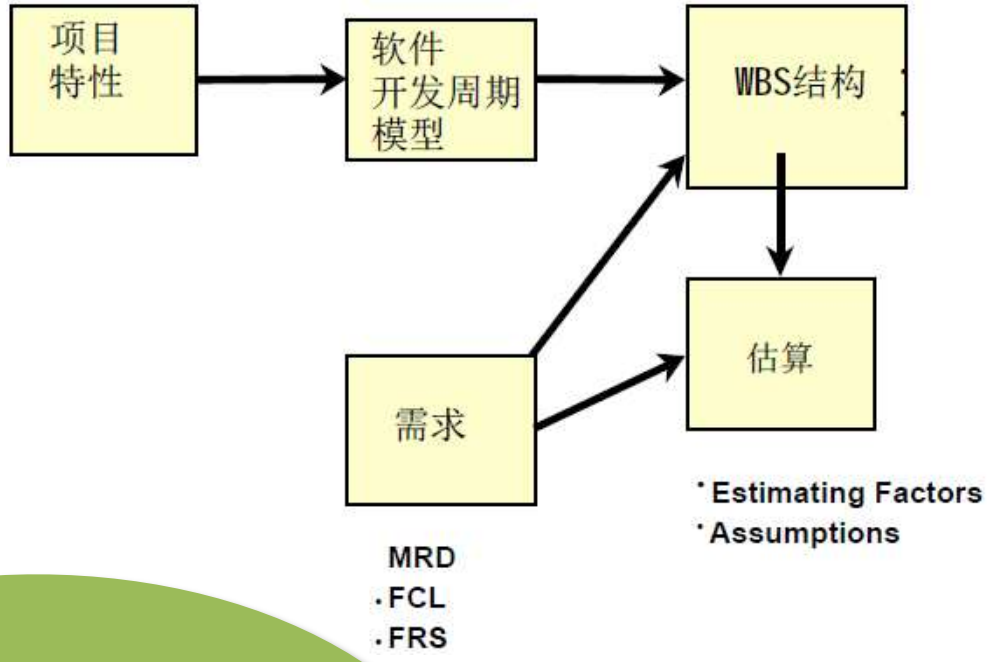
项目→任务→工作→日常活动

WBS分解的标准：

- 分解后的活动结构清晰
- 逻辑上形成一个大的活动
- 集成了所有的关键因素包含临时的里程碑和监控点
- 所有活动全部定义清楚。

影响估算的因素

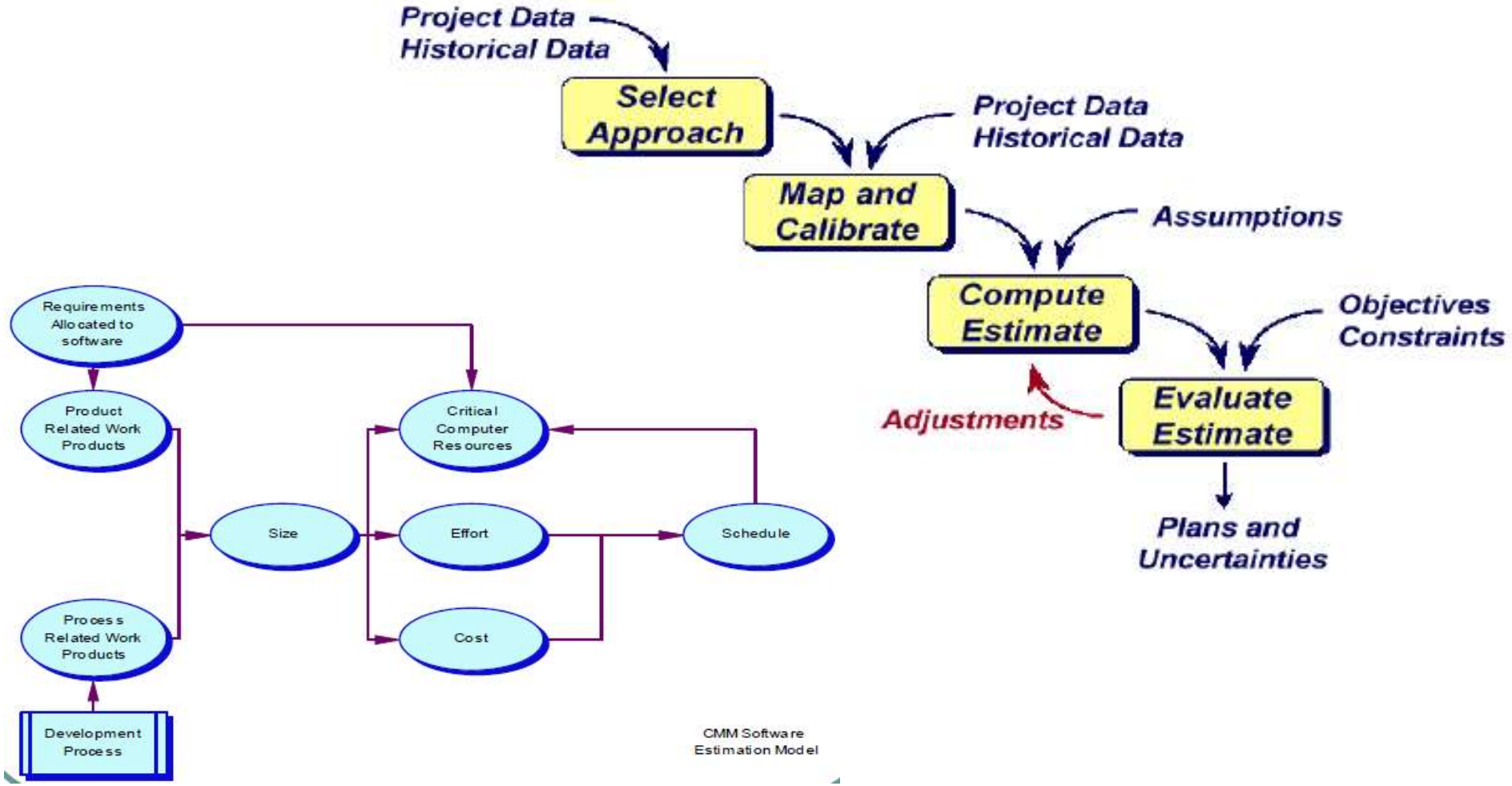




谁来做估算？

- 估算小组成员
- 项目经理
- 估算协调员(可选)
- SQA工程师
- SEPG
- 管理部门(软件或非软件)

估算的过程



CMM软件估算模型

何时做估算？

- 一个软件生命期中至少做三次估算：
  ① 在需求或建议书即将被批准的时候，覆盖系统设计直至项目提交(估算基于需求和计划中的任务)
  ② 在设计即将被批准的时候，覆盖系统实施直至项目提交(估算基于项目当前数据和更细化的项目任务拆分)
  ③ 在系统实施将要完成的时候，覆盖集成测试直至项目提交(估算基于项目当前数据和更细化的项目任务拆分)
- 需求变更较大时以及各阶段结束时的管理复审，要对下一阶段的估算进行修正。

软件估算方法可分为两大类：

- 基于经验
  - 估计过程中不涉及显式的建模活动，也不借用其他估计模型或工具的支持。
  - 在实践中往往由多个专家组成估计小组，采用Delphi，WideBand Delphi，组讨论等技术得出收敛的估计值。
  - 基于经验的方法应用最为广泛，但准确性的高低完全依赖估计者的个人能力，这不仅不利于组织的知识积累与经验复用，也无法对估计结果进行有效性验证。
- 基于模型
  - 当前估计领域的热点，已出现了一大批成本估计模型，如COCOMO模型、基于神经网络的模型、基于类比技术的模型等。
  - 市场上也有多种商业化的估计工具，如COSTAR，ESTIMACS，SPQR/20，SEER-SEM，SLIM-suit等。实践表明，目前尚无一种模型在任何环境下都能提供最佳估计。

基于模型的软件成本估计方法分为3类：

- 算法驱动式模型
  - 基本思想：影响软件项目的因素（例如：产品的复杂性、人员的能力、平台的经验）与软件项目的指标（例如：工作量、工期）之间存在着可用公式表示的确定关系。
  - 例子较多，如：FP、SLIM、COCOMO、PRICE-S、COCOTS、Doty、IBM-FSD、Boeing模型等。
- 数据驱动式模型
  - 基于统计技术的模型：以数据分析为基础，需要依据大量的历史数据建模。常用的统计建模技术有普通最小平方回归（OLS）、强回归（RR）、逐步方差分析（Stepwise ANOVA）等。
  - 基于机器学习技术的模型：根据大量的历史数据，采用某种学习算法构造能够拟和数据的规则。典型技术有OSR（optimized set reduction）、分类回归树（CART）、人工神经网络等。
- 复合式模型
  - 典型方法有Bayesian方法、类比方法、COBRA方法。

估算中存在的问题

- 没有历史数据可以参照
- 估算目标的度量单位选择不合适
- 有低估的倾向，估计值只是实际值的1/2或更低
- 估算准确与需求不定之间带来的矛盾
- 估算中的误区

估算准确性不高的主要原因：

①软件是不可见的；

②每个软件项目都是独一无二的；

③现实的估计必须考虑项目的约束、可用的资源、人员的技能等因素，进一步增加了估计的复杂性。

估算的偏见与避免的方法

自我的作用

- 偏见使人们更倾向于低估所需的时间
- 当自己参与某项任务时，你不能合理地估计完成该任务所需的时间。
- 当你很在意结果的时候，你估算的质量接近于0。

避免的方法

- 采用合理的估算技术
- 使用独立的估算者
- 依靠实际数据反馈，做调整

不确定/不可知因素

避免的方法

- 利用历史数据修正
- 多重估算，并进行估算结果的归纳
- 估算结果的平均值
- 加权平均数
- 修正——迭代式估算

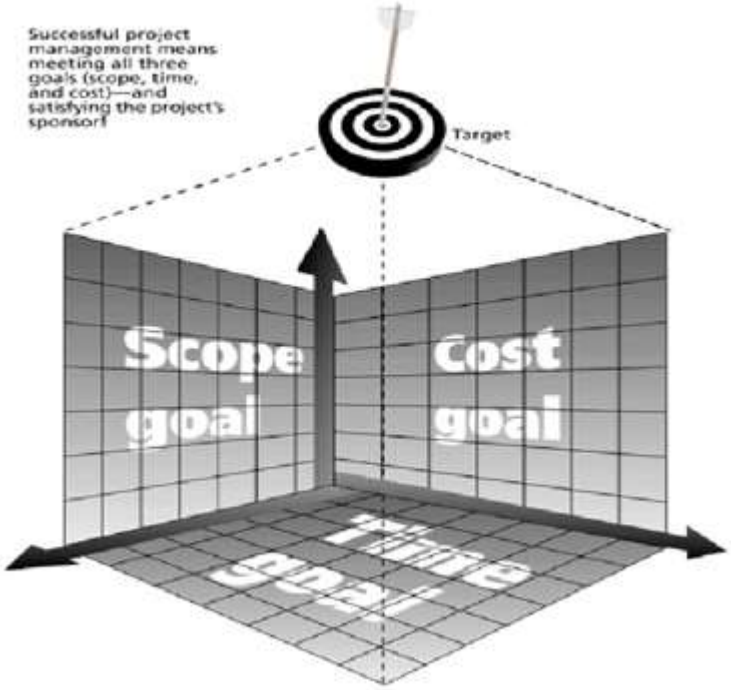
估算的基本原则



- 尽可能采用数学估算模型；
- 估算应该基于历史数据进行推断；
- 当不具备以上两种条件时，估算要依据专家的判断进行推断；
- 为保证估算结果的合理性，要用多种方法对同一个估算对象进行估算；
- 估算对象（产品、活动）一定要分解为合适的颗粒度，以便于估算；
- 估算中用到的假设、方法、技术以及实践数据一定要用文档记录；
- 历史数据选择原则：相似项目并陈述选择的理由；
- 估算结果要文档化、评审，并被相关的小组或人员认可；
- 项目整个过程中，估算结果要不断得到修正或精化，来反映开发者对项目深入的理解和认识，估算文档要根据项目过程现状进行复审。

2.3.2 规模估算问题

- 软件规模度量(size measurement)是估算软件项目工作量、编制成本预算、策划合理项目进度的基础。规模度量是软件项目失败的重要原因之一。
- 软件大小（size）是一个没有意义的衡量尺度，一般进行代理测量。
- 可以度量软件规模方面：
  - 软件功能数量： 数据流图、用例图
  - 软件模块数量：模块功能结果图
  - 代码的行数：操作符、操作数
  - 设计文档的页数
  - 用户手册的页数



常见的规模估算方法

- 功能点分析（FPA：function points analysis）
- 代码行数（LOC：lines of code）
- 德尔菲法（Delphi technique）
- COCOCO模型
- 特征点（feature point）
- 对象点（object point）
- 3-D功能点（3-D function points）
- Bang度量（DeMarco’s bang metric）
- 模糊逻辑（fuzzy logic）
- 标准构件法（standard component）

编程语言	SLOC/Albrecht 功能点
汇编语言	320
C	150
Query Languages	16
Spreadsheets languages	6

组件	SLOC/ 组件
SLOC	1
对象指令	0.28
文件	2535
报告	967
屏	818

例：估算单位样例

工作产品与活动	估算单位样例（估算数值）
需求文档	➢ 页数 ➢ “项目应”的个数 ➢ 功能点的个数（查询树， 输入树， 输出树， 外部数据来源） ➢ 应用实例的个数（Use Case个数）
设计文档	➢ 应用实例的个数 ➢ 屏幕的个数 ➢ 对象个数，方法个数，关系个数 ➢ 报告数 ➢ 模块数
编程 ， 单元测试	➢ 模块数 ➢ 程序行数 ➢ 数据库表数， 列数，字段数，存储过程数， 等等。 ➢ 窗口数， 对话框数 ➢ 个人报告数 ➢ 过程数
整合测试	➢ 测试案例 ➢ 测试实例 ➢ 测试数据集
同行评审	➢ 参与人数
收集需求	➢ 用户个数
电话调查	➢ 通话人次 ➢ 问题数及问题复杂度
用户交互	➢ 用户数 ➢ 访问地点数

需求变更带来的估算问题：

- 需求随着时间改变，估算必须与需求保持一致，尽一切可能满足项目的进度安排和预算。
- 估算小组使用需求信息估算软件产出物的大小，进而估算所需人时。因此，不准确不完整的需求信息会造成估算不准，即便使用最好的估算流程和工具。

2.3.3 代码行技术

- 根据以往的经验和历史数据，估计实现一个功能需要的源程序行数的方法。
- 当有类似项目的历史数据可供参考时，这种方法估计出的数据还是比较准确。
- 估算方法：
  - 为了使估计值更接近实际值，可以由多名有经验的软件工程师分别作出估计；
  - 每个人都估计程序的最小规模(a)、最大规模(b)和最可能的规模(m)，分别算出这三种规模的平均值a、b和m之后，再用下式计算程序规模的估计值：

$$L = \frac{\bar{a} + 4\bar{m} + \bar{b}}{6}$$

面向规模的度量数据样例

项目名称	代码行 (KLOC)	工作量 (人月)	成本 (千元)	文档页数	错误 (发布前)	缺陷 (一年内)	人数
项目1	121	24	168	365	134	29	3
项目2	272	62	440	1224	321	86	5
项目3	202	43	314	1050	256	64	6

例1：基于LOC估算

软件描述：

- CAD软件，接收来自工程师输入的3D或2D几何图形。工程师通过用户界面和CAD软件进行交互，并控制它，该界面应当表现出良好的人机界面设计的特征。所有几何数据及其他支持信息都保存在一个CAD数据库中。
- 要求开发设计分析模块，以产生必要的输出，这些输出将表现在不同的图形设备上。软件在设计中要考虑与外设交互并控制它们。
- 除显示器之外，外设包括鼠标、数字化仪和激光打印机。

假设已经对上述要求进行了求精和分解，界定了以下的主要软件子功能：

(1) 用户界面及控制机制
(2) 二维几何分析(2DGA)
(3) 三维几何分析(3DGA)
(4) 数据库管理(DBM)
(5) 计算机图形显示机制(CGDF)
(6) 外设控制(PC)
(7) 设计分析模块(DAM)

估算过程：

- 利用本组织的度量基线数据，遵照LOC的“三点”估算技术，对分解后的需求进行LOC估算。
- 例如，对于3DGA功能：a=4600；m=6900；b=8600。

$$L = (a + 4m + b)/6 = 6800LOC$$

- 类似地，求出其他被分解部分的LOC期望值，形成表。

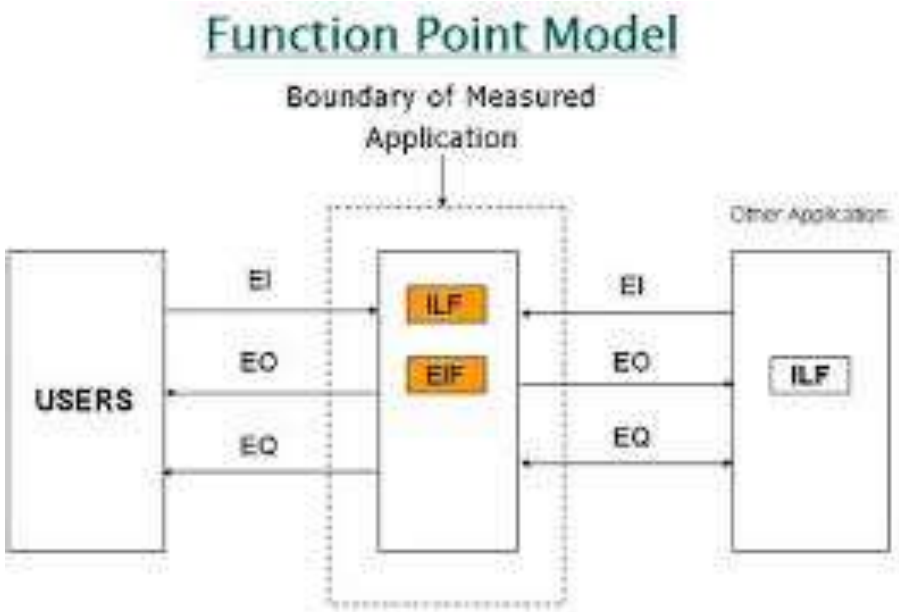
基于问题分解的LOC方法的估算表

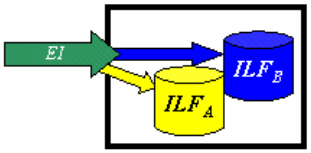
问题分解所得的子功能	估算的LOC期望值(行)
用户界面及控制机制	2300
二维几何分析(2DGA)	5300
三维几何分析(3DGA)	6800
数据库管理(DBM)	3350
计算机图形显示机制(CGDF)	4950
外设控制(PC)	2100
设计分析模块(DAM)	8400
估算总代码行数期望值	33200

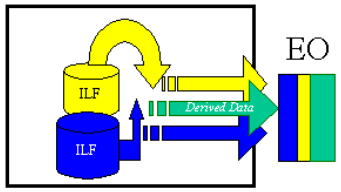
- 查询本组织的度量基线，得估算结果：
  - 此类系统的平均生产率为620*LOC/PM*；
  - 平均人月成本为8000美元/人月，则LOC平均成本为13*美元/LOC*。
  - 计算可知本项目的总成本为431000美元，总工作量54个人月。
  - 如果人力资源投入6名合格的工程师，则预计工期为9个月。

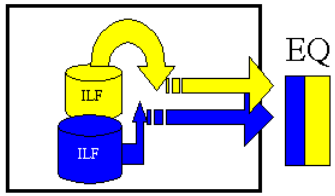
2.3.4 功能点技术

- 以功能点(FP，Function Point)为单位，度量软件的规模。
- 功能点技术依据对软件信息域特性和软件复杂性的评估结果，估算软件规模。









1. Data Functions - Internal Logical Files (ILFs)
2. Data Functions - External Interface Files (EIFs)
3. Transaction Functions - External Inputs (EI's)
4. Transaction Functions - External Outputs (EO's)
5. Transaction Functions - External Inquiries (EQ's)

估算功能点的步骤

(1) 计算未调整的功能点数*UFP*

- 首先，把产品信息域的每个特性都分类成简单级、平均级或复杂级。根据其等级，为每个特性都分配一个功能点数；
- 然后，用下列公式计算未调整的功能点数UFP

$$UFP=a_1\times Inp+a_2\times Out+a_3\times Inq+a_4\times Maf+a_5\times Inf$$

- 其中，a_i(1≤i≤5)是信息域特性系数，其值由相应特性的复杂级别决定，参见表1。

表1　信息域特性系数值

复杂级别 / 特性系数	简单	平均	复杂
输入系数 a1	3	4	6
输出系数 a2	4	5	7
查询系数 a3	3	4	6
文件系数 a4	7	10	15
接口系数 a5	5	7	10

（续）

功能点估算法之调整因子.docx

(2) 计算技术复杂性因子 *TCF*

- 度量14种技术因素对软件规模的影响程度。这些因素包括高处理率、性能标准，参见表2，并用F_i(1≤i≤14)代表这些因素。
- 根据软件特点，为每个因素分配一个从0(不存在或对软件规模无影响)到5(有很大影响)的值。
- 然后，用下式计算技术因素对软件规模的综合影响程度 *DI*：

$$DI = \sum_{i=1}^{14} F_i$$

表 2 技术因素

序号	F_i	技术因素
1	F1	数据通信
2	F2	分布式数据处理
3	F3	性能标准
4	F4	高负荷的硬件
5	F5	高处理率
6	F6	联机数据输入
7	F7	终端用户效率
8	F8	联机更新
9	F9	复杂的计算
10	F10	可重用性
11	F11	安装方便
12	F12	操作方便
13	F13	可移植性
14	F14	可维护性

（续）

- 技术复杂性因子*TCF*由下式计算：

$$TCF = 0.65 + 0.01 \times DI$$

- 因为*DI*的值在0～70之间，所以*TCF*的值在0.65～1.35之间。

(3)计算功能点数*FP*

- 功能点数FP由下式计算：

$$FP = UFP \times TCF$$

例:基于FP估算

对于例1（ CAD软件）中的各个子功能进一步细化，将所有功能都分解为EI、EO、EQ以及ILF、EIF的组合，假设加权因子都取为“平均”，计算出各类功能点表。

估算信息域值——基于问题分解的FP估算

信息域值	乐观值	可能值	悲观值	估算计数	加权因子	FP计数
EI	20	24	30	24	4	96
EO	12	15	22	16	5	80
EQ	16	22	28	22	4	88
ILF	4	4	5	4	10	40
EIF	2	2	3	2	7	14
总计数值						318

（续）

CAD软件项目复杂度调整

因子	值	因子	值
备份和复原	4	信息域值复杂度	5
数据通信	2	内部处理复杂度	5
分布式处理	0	可复用需求	4
关键性能	4	设计中的转换及安装	3
现有的操作环境	3	多次安装	5
联机数据登录	4	方便修改的应用设计	5
多屏幕输入切换	5	复杂度调整因子	1.17
主文件联机更新	3		

- 最后，得到整个项目的FP估算期望值：

$$FP = 318 \times [0.65 + 0.01 \times \sum Fi] = 372(FP)$$

- 查询组织的度量基线数据库得知：
  - **这类系统的平均生产率为6.5*FP/PM***
  - **一个PM的成本仍取8000美元**
  - **每个FP的平均成本约为1230美元**
  - **总项目成本估算约为457000美元**
  - **工作量估算期望值是58个人月**

2.3.5 工作量估算

- 软件估算模型使用由经验导出的公式来预测软件开发的工作量，工作量是软件规模(*LOC*或*FP*)的函数，工作量的单位通常是人月(*man month*)。
- 支持大多数估算模型的经验数据，都是从有限个项目的样本集中总结出来的，因此，没有一个估算模型能够适用于所有类型的软件和开发环境。
- 工作量估算有两类模型：
  - 静态模型
  - 动态模型

静态单变量模型

- 这类模型的总体结构形式如下：

$$E=A+B\times(ev)^C$$

- 其中：
  - E是以人月为单位的工作量
  - A、B和C是由经验数据导出的常数
  - ev是估算变量(LOC或FP)

静态模型举例：

- 面向*LOC*的估算模型

  (1) Walston Felix模型：$E=5.2\times(KLOC)^{0.91}$

  (2) Bailey Basili模型：$E=5.5+0.73\times(KLOC)^{1.16}$

  (3) Boehm简单模型：$E=3.2\times(KLOC)^{1.05}$

  (4) Doty模型(在KLOC＞9的情况下)：$E=5.288\times(KLOC)^{1.407}$

- 面向*FP*的估算模型

  (1) Albrecht & Gaffney模型：$E=-13.39+0.0545\times FP$

  (2) Kemerer模型：$E=60.62\times7.728\times10^{-8}\times FP^3$

  (3) Maston、Barnett和Mellichamp模型：$E=585.7+5.12\times FP$

动态多变量模型

- 动态多变量估算模型的形式如下：

$$E=(LOC\times B^{0.333}/P)^3\times(1/t)^4$$

- 其中：
  - E 是以人月或人年为单位的工作量；
  - t 是以月或年为单位的项目持续时间；
  - B 是“特殊技术因子”，它随着对集成、测试、质量保证、文档及管理技术的需求的增长而缓慢增加，对于较小的程序(*KLOC*=5～10)，B=0.16，对于超过70*KLOC*的程序，B=0.39；
  - P 是“生产率参数”，它反映了下述因素对工作量的影响：
    - 总体的过程成熟度及管理水平；
    - 使用良好的软件工程实践的程度；
    - 使用的程序设计语言的级别；
    - 软件环境的状态；
    - 软件项目组的技术及经验；
    - 应用系统的复杂程度。

- P典型值举例
  - 实时嵌入式软件时，P=2000；
  - 电信和系统软件时，P=10000；
  - 商业系统应用软件时，P=28000。
  - 适用于当前项目的生产率参数，可以从历史数据导出。
  - 开发同一个软件(即LOC固定)的时候，如果把项目持续时间延长一些，则可降低完成项目所需要的工作量。

COCOMO模型

- COCOMO（COnstructive COst MOde，构造性成本模型）是由Boehm提出的最精确、最易于使用的软件成本估算模型方法之一。
- 该模型分为：
  - 基本COCOMO模型
  - 中级COCOMO模型
  - 详细COCOMO模型

基本COCOMO模型

- 它是一个静态单变量模型，对整个软件系统进行估算。

$$E = a_b(KLOC)\exp(b_b)$$

$$D = c_b(E)\exp(d_b)$$

- 其中：
  - E 为开发所需的人力（人-月）
  - D 为所需的开发时间（月）
  - $KLOC$ 为估计提交的代码行
  - a_b、b_b、c_b和d_b 是指不同软件开发方式的值，见下表。

表 基本COCOMO模型

方式	a_b	b_b	c_b	d_b
组织模式	2.4	1.05	2.5	0.38
半独立模式	3.0	1.12	2.5	0.35
嵌入模式	3.6	1.2	2.5	0.32

（续）

- 软件开发项目可以分成三种模式：
  - 组织式：对组织式软件的要求通常不苛刻，开发人员经验丰富，而且对软件的使用环境很熟悉(通常是为自己所在的组织开发软件)，程序规模一般不大(小于50,000行代码)。例如，简单的商业数据处理系统即属于组织式。
  - 嵌入式：嵌入式软件需要在很强的约束条件下运行，通常和某些硬设备紧密结合在一起(例如，空中交通管理软件)，对这类软件系统的要求通常十分苛刻。
  - 半独立式：对半独立式软件的要求通常介于上述两类软件之间，但是这类软件的规模一般都比较大(可达300,000行代码)。

中级COCOMO模型

- 它是静态多变量模型，将软件系统模型分为系统和部件两个层次，系统是由部件构成的，它把软件开发所需人力（成本）看作是程序大小和一系列“成本驱动属性”的函数，用于部件级的估算，更精确些。
- 该模型把软件开发工作量表示成代码行数（*KLOC*）的非线性函数：

$$E = C_1 \times KLOC^a \times \prod_{i=1}^{15} f_i$$

- 其中：

E 是开发工作量(以人月为单位)

C_1 是模型系数

$KLOC$ 是估计的代码行数(以千行为单位)

a 是模型指数

f_i (i=1到15)是成本因素

表 中级COCOMO模型

方式	c_i	f_i
组织模式	3.2	1.05
半独立模式	3.0	1.12
嵌入模式	2.8	1.2

成本因素f_i

- 每个f_i都根据它的重要程度和影响大小赋予一定数值。
- f_i影响所有工程项目，即使不使用*COCOMO*模型估算成本，也应该注意这些因素。可以把成本因素划分成产品因素、计算机因素、人员因素和项目因素等，下面简单介绍主要的成本因素。
- f_i的主要的成本因素：

1. 产品因素
   (1) 要求的软件可靠性(RELY)
   (2) 数据库规模(DATA)
   (3) 软件产品复杂程度(CPLX)

f_i的主要的成本因素（续）

2. 计算机因素
   (1) 执行时间的约束(TIME)
   (2) 存储约束(STOR)
   (3) 环境变更率(VIRT)
   (4) 计算机换向时间(TURN)
3. 人员因素
   (1) 系统分析员的能力(ACAP)
   (2) 应用经验(AEXP)
   (3) 程序员的能力(PCAP)
   (4) 环境知识(VEXP)
   (5) 语言知识(LEXP)
4. 项目因素
   (1) 程序设计实践(MODP)
   (2) 软件工具(TOOL)
   (3) 进度约束(SCED)

影响软件开发成本的工作量系数

表 影响软件开发成本的工作量系数

成本因素	级别					
	甚低	低	正常	高	甚高	特高
RELY	0.75	0.88	1.00	1.15	1.40	
DATA		0.94	1.00	1.08	1.16	
CPLX	0.70	0.85	1.00	1.15	1.30	1.65
TIME			1.00	1.06	1.21	1.56
STOR			1.00	1.06	1.21	1.56
VIRT		0.87	1.00	1.15	1.30	
TURN		0.87	1.00	1.07	1.15	
ACAP	1.46	1.19	1.00	0.86	0.71	
AEXP	1.29	1.13	1.00	0.91	0.82	
PCAP	1.42	1.17	1.00	0.86	0.70	
VEXP	1.21	1.10	1.00	0.90		
LEXP	1.14	1.07	1.00	0.95		
MODP	1.24	1.10	1.00	0.91	0.82	
TOOL	1.24	1.10	1.00	0.91	9.83	
SCED	1.23	1.08	1.00	1.04	1.10	

详细COCOMO模型

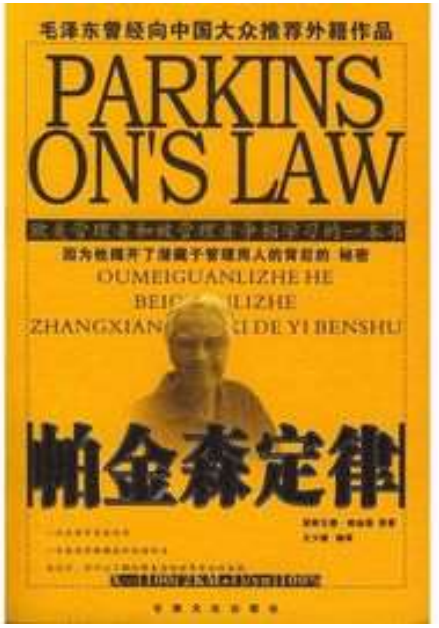
- 详细模型将软件系统模型分为3个层次：
  - 系统
  - 子系统
  - 模块
- 它的进度计算公式与中级模型相同，但是调整方式有区别。不再使用统一的工作量调整因子表，而是将15项工作量调整因子按照不同工作阶段(分析与高层设计、详细设计、编码与单元测试、集成及测试)分别考虑，给出不同的阶段工作量调整数据表，最后使用各个阶段工作量调整因子的综合均值进行工作量和工作进度调整，更为贴近实际。

第2讲　软件项目管理

2.4 项目进度安排

Parkinson's Law

Originally, Parkinson's law is the adage that "work expands so as to fill the time available for its completion", and the title of a book which made it well-known. However, in current understanding, Parkinson's law is a reference to the self-satisfying uncontrolled growth of the bureaucratic apparatus in an organization.



Prof. Cyril Northcote Parkinson

书中的例子：如果分配一个人6个小时去完成一个只需6分钟的工作，那么这个工作会在6个小时完成。

项目管理中考虑帕金森定律带来的影响：

- 正确估算
- 日常开发管理中，对时间表有预留缓冲，以应对突发状况。

时间管理-重要与紧急



- 重要且紧急
  - 紧急状况
  - 迫切的问题
  - 限期完成的工作
  - 你不做其他人也不能做
- 重要不紧急
  - 准备工作
  - 预防措施
  - 价值观的澄清
  - 计划
  - 人际关系的建立
  - 真正的再创造
  - 增进自己的能力

- 紧急不重要
  - 造成干扰的事、电话
  - 信件、报告
  - 会议
  - 许多迫在眉睫的急事
  - 符合别人期望的事
- 不重要不紧急
  - 忙碌琐碎的事
  - 广告函件
  - 电话
  - 逃避性活动
  - 等待时间

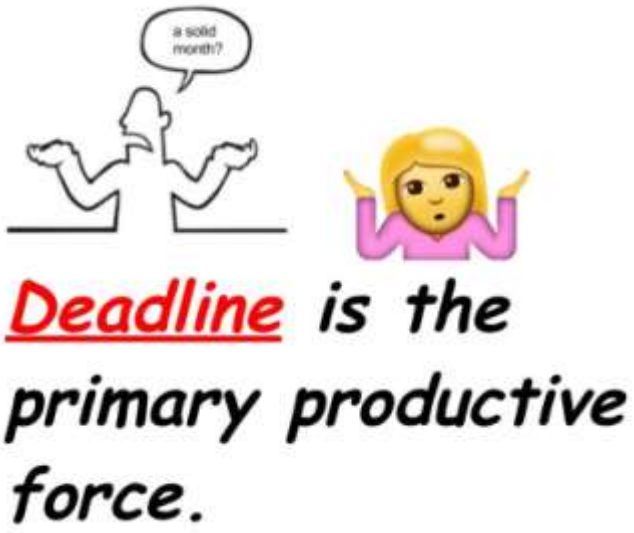
进度安排的目标

- 软件项目的进度安排是一项活动，它通过把工作量分配给特定的软件工程任务，并规定完成各项任务的起、止日期，从而将估算的工作量分布于计划好的项目持续期内。
- 进度安排的目标是：
  - 定义全部项目任务
  - 识别出关键任务
  - 跟踪关键任务的进展状况
  - 保证能及时发现拖延进度的情况

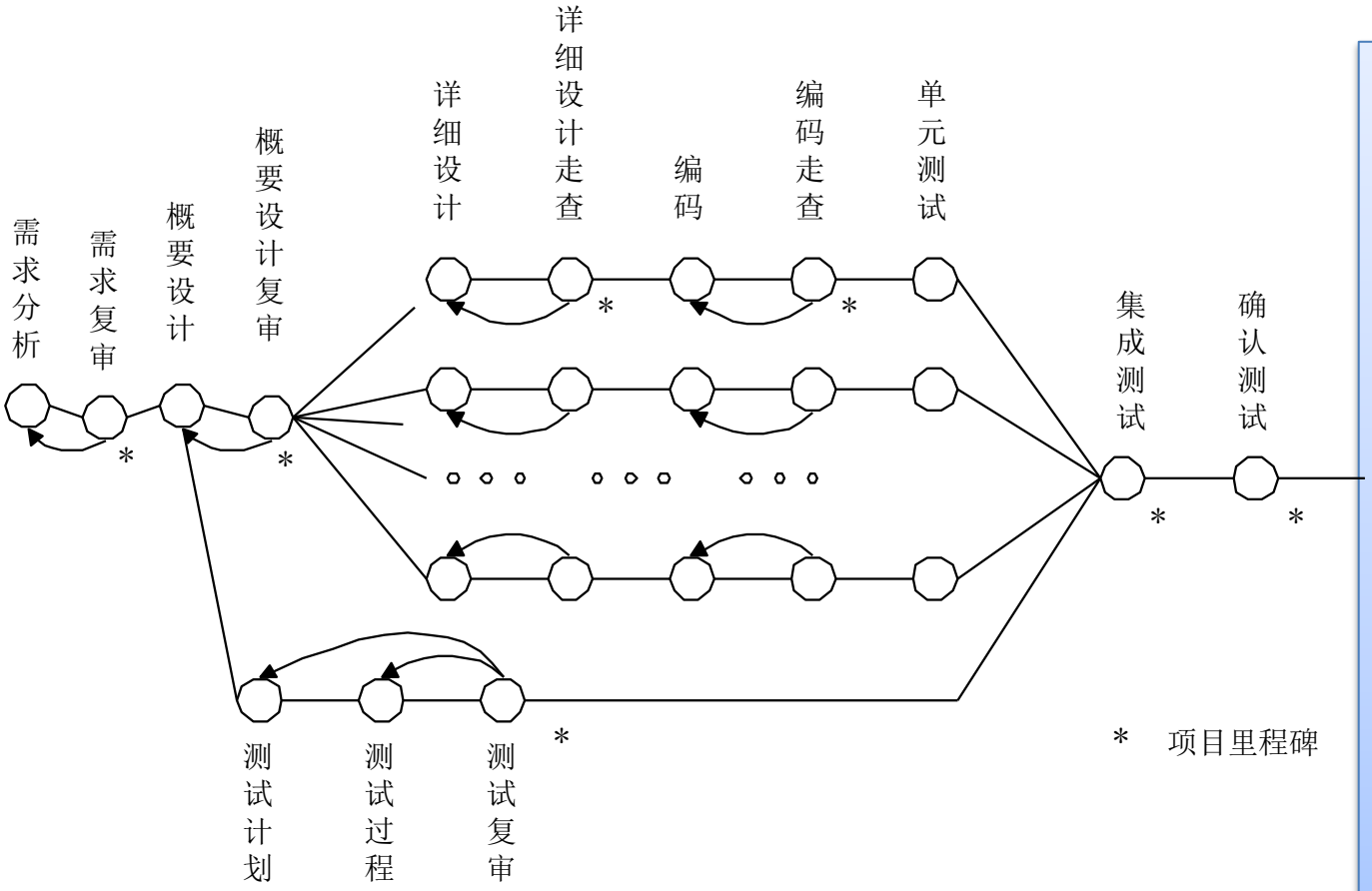
布鲁克定律：

“在一个已经延期的项目中增加人手只会让项目延期更长”。

(这个定律来自《人月神话》)



典型软件开发任务的并行



由于软件工程活动的并行性，制定开发进度计划和制定任务之间的依赖关系是十分重要的。

项目经理必须了解处于关键路径上的任务进展的情况，如果这些任务能及时完成，则整个项目就可以按计划完成。

进度安排中的问题

- 问题的难度和开发成本估算同样困难。
- 生产率与人数不成正比。
- 增加人到一个延期的项目中，由于通讯的代价增加，可能使项目延期更多。
- 总是发生不可预料的事情，因此计划中要考虑偶发事件的发生。

进度安排的基本原则

1. 任务划分
2. 相互依赖性
3. 时间分配
4. 工作量确认
5. 定义责任
6. 定义结果
7. 定义里程碑

帕累托谬论

“当你完成了80%的工作，你会认为真的只剩下20%的工作要做”。但你可能低估了剩下的20%工作，因为它可能占用你80%的时间。

2.4.1 甘特图（Gantt）

- Gantt图是历史悠久、应用广泛的进度计划工具。
- 它是先把任务分解成子任务，然后用水平线段来描述各个任务及子任务的进度安排。

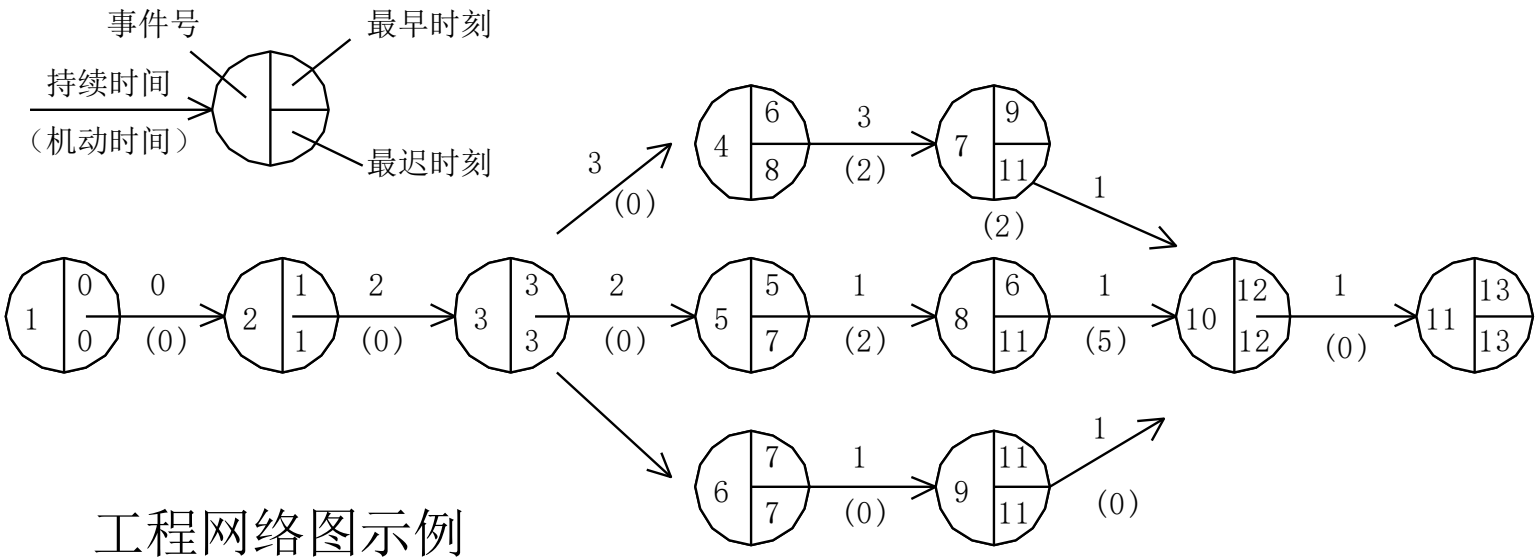
进程计划时间表

时间（月） 项目	1999年									
	3月	4月	5月	6月	7月	8月	9月	10月	11月	12月
前期准备										
系统调查										
系统分析										
系统设计										
系统实施										
系统试运行										
系统测试										
系统验收										
系统正式运行										

- 不能显式地描绘各项作业彼此间的依赖关系；
- 进度的关键部分不明确，难于判定哪些部分应当是主攻和主控的对象；
- 计划中有潜力的部分及潜力的大小不明确，往往造成潜力的浪费；
- 只能表示任务之间的并行与串行的关系，难以反映多个任务之间存在的复杂关系，不能直观表示任务之间相互依赖制约关系，以及哪些任务是关键子任务等信息，因此仅仅用Gantt图作为进度的安排是不够的。

2.4.2 工程网络图

- 工程网络图，是一种有向图，是制定进度计划时常用的图形工具。
- 它能描绘任务分解情况以及每项作业的开始时间和结束时间，此外，它还显式地描绘各个作业彼此间的依赖关系。



工程网络图示例

估算进度方法

- 首先，把每个作业估计需要使用的时间写在表示该项作业的箭头上方。
- 其次，为每个事件计算下述两个统计数字：最早时刻EET和最迟时刻LET。这两个数字将分别写在表示事件的圆圈的右上角和右下角。
- 关键路径：
  - 事件的最早时刻和最迟时刻相同，这些事件组成关键路径。
  - 关键路径上的事件(关键事件)必须准时发生，组成关键路径的作业(关键作业)的实际持续时间不能超过估计的持续时间，否则工程就不能准时结束。
- 机动时间：
  - 不在关键路径上的作业有一定程度的机动余地。

第2讲 软件项目管理

2.5 软件度量

主要内容

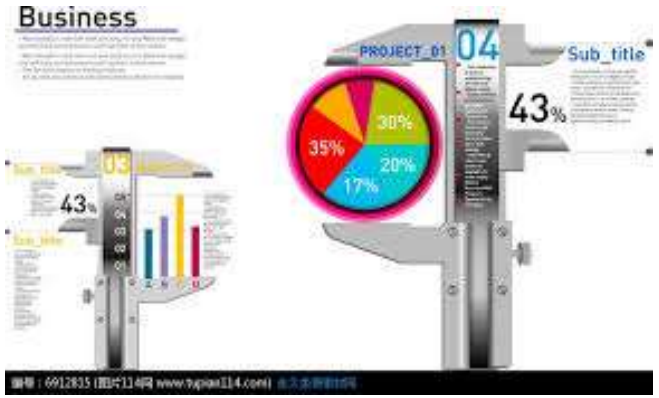
- 为什么度量
- 什么是度量
- 如何进行度量
- 度量对工作的影响



度量的意义

不能度量，就无法控制，也就无法管理。

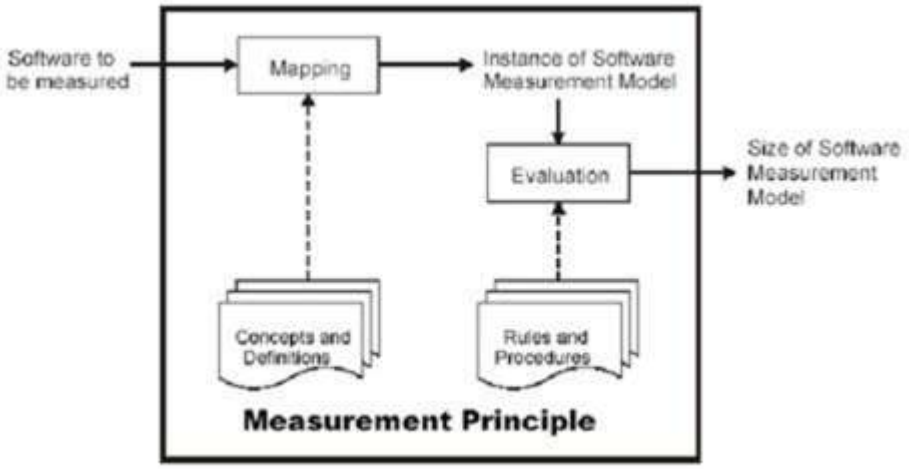
——If you can’t measure it, you can neither control nor manage it.



2.5.1 什么是度量

- 度量：
  - 根据一定的规则，将数字或符号赋予系统、构件、过程等实体的特定属性，从而能清晰地理解该实体及其属性。
  - 简而言之，度量就是对事物属性的量化。

- 度量实质
  - 度量是一个函数，它的输入是软件数据，输出是单一的数值，能用以解释软件所具有的一个给定属性对软件质量和过程影响的程度。
  - 任意一个度量都必须具备4个因素：
    - 被度量的对象
    - 将被度量的属性
    - 赋予对象的数值或符号
    - 使对象与数值或符号相联系的映射



Measurement principle of functional size measurement

度量的重要性

- 度量活动可以对软件开发项目状态和产品质量给予量化的表示，为加强和改进研发工作提供详细的指导。
- 可以积累数据，为今后的估算提供参考。
- 度量的目的
  - 理解——获得对过程、产品、资源、环境的理解，确定以后预测的基线和模型。这是评估、预测、改进活动的基础。
  - 预测——通过理解过程、产品各要素之间的关系建立模型，由已知的要素推算、估计其它要素，以例合理分配资源、合理制定计划。
  - 评估——分析活动与计划的符合程度，确定是否有偏差，以便控制其评估最终产品的质量，评估新技术的影响，评估过程改进对过程和产品的影响。
  - 改进——根据得到的量化信息，可以帮助识别障碍物、查找问题根源，以及能提高产品质量和过程效率的其它方法。与以前的量化信息进行比较，可以证实这些方法是否有效。

度量分类

- 过程度量
  - 对于软件开发过程的度量，目的是形成组织的各种模型，作为对项目、产品的度量基础；以及对软件开发过程进行持续改进，提高软件生产力。
  - 过程度量往往不是直接进行，而是通过大量的项目度量分析、总结得出来间接实现。
- 项目度量
  - 对于软件开发项目的特定度量，目的是评估项目开发过程中的质量，预测项目进度、工作量等，辅助管理者进行质量控制和项目控制。
- 产品度量
  - 是对项目开发结果——最终产品的度量。
  - 一般来说，产品度量指的是对产品的质量度量。

项目度量的内容

- 规模度量
- 工作量度量
- 进度度量
- 生产力度量
- 风险度量
- 其它的项目动态度量（如：需求变动、代码动态增长等）

过程度量与项目度量的区别

- 过程度量是战略性的，针对组织范围内进行，是组织大量项目实践的总结和模型化，对于项目度量提供指导意义；
- 项目度量是战术性的，针对具体的项目进行，预测、评估、改进项目工作。产品度量是对产品质量的度量，用于对产品质量的评估和预测。

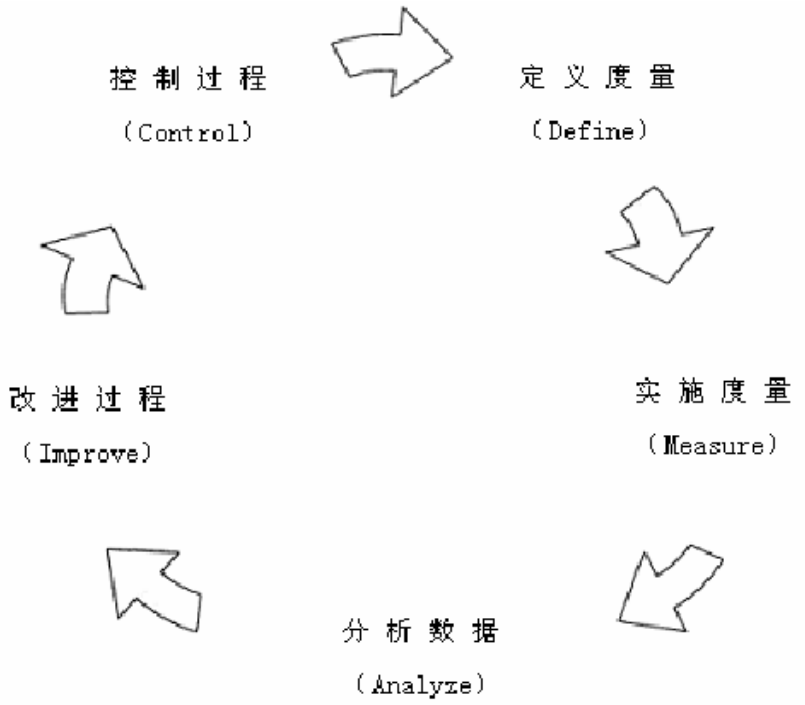
一些常用度量的意义

- 缺陷度量
  - 帮助确定产品缺陷变化的状态，并指示缺陷活动所需要的工作量；
  - 帮助分析产品缺陷分布情况，并指示需要加强何种研发活动，需要何种技能培训；
  - 帮助产品的遗留缺陷情况，预测产品发布后缺陷的影响情况。
- 规模度量
  - 帮助预测完成产品所代码量。规模度量是工作量度量、进度度量的基础，通过规模度量，可以更合理地安排资源和人力，预测合理的工作量，监控研发进度的正常进行。
- 复杂度度量
  - 复杂度度量帮助预测软件产品各部分的复杂性，以便选择最可靠的程序设计部分，更合理地分配研发资源和人力，获得最高的效益。

5.2.2 如何进行度量活动

目标驱动的度量活动主要包括5个阶段：

① 识别目标
② 定义度量过程
③ 数据收集
④ 数据分析与反馈
⑤ 过程改进



软件开发主体及其度量内容

经营者
开发组织

- 顾客满意度
- 收益
- 风险
- 绩效
- 发布的缺陷的级别
- 产品开发周期
- 日程与作业量估算精度
- 复用有效性
- 计划与实际的成本

管理者
项目组

- 不同阶段的成本
- 不同开发小组的生产率
- 产品规模
- 工作量分配
- 需求状况
- 测试用例合格率
- 主要里程碑之间的估算期间与实际期间
- 估算与实际的员工水平
- 集成测试和系统测试检出的缺陷数目
- 审查发现的缺陷数目
- 缺陷状况
- 需求稳定性
- 计划和完成的任务数目

作业者
软件开发人员

- 工作量分配
- 估算与实际的任务期间与工作量
- 单元测试代码覆盖率
- 单元测试检出缺陷数目
- 代码和设计的复杂性

度量取向

- 度量取向是软件开发诸多事项的横断面，包括顾客满意度度量、质量度量、项目度量、以及品牌资产度量、知识产权价值度量等。
- 度量取向依靠事实、数据、原理、法则；方法是测试、审核、调查；工具是统计、数字、模型；其标准是量化的指标。



度量策略

- 度量对象的属性可以通过度量此对象的其他一些属性进行计算而得，这样的度量称为**间接度量**。
- 软件的属性可分为两种：
  - 内部属性
    - 通常描述软件结构上的复杂性，如软件大小、控制流、耦合度等。
    - 一般有清晰的定义并能进行客观的度量。
    - 度量研究的任务之一就是建立外部属性和内部属性之间的合理联系。
  - 外部属性
    - 通常涉及到人和环境等外部因素，如复杂性、可维护性、可读性等。
    - 通常只有外部属性的度量才能真正给出人们所需的可靠数据，并且预示软件开发行为。
    - 外部属性常常缺乏清晰的定义，也无法进行直接的客观度量，所以一般只能通过内部属性进行推测性的间接度量来得出外部属性值。

2.5.3 度量对工作有什么影响

- 根据度量目标和要求的不同，度量活动可能涉及一个项目的所有人员，也可能包括各种活动。
- 度量的范围
  - 度量会涉及参与项目每个人和它们的工作：
    - 从需求分析到设计、实现、测试和软件维护
    - 从项目管理者到开发者、测试者、技术支持者，甚至用户。
    - 从代码实现到各种评审、检查。
  - 每一个阶段、每一个角色的各种软件活动都会被纳入软件度量活动的范围中。

可以度量个人吗？

“正好用这个度量结果来评价下属的工作绩效”
“软件度量会评价我的工作绩效吗？”
“我提供的数据会不会用来作为评价我的依据？”

度量从不用于评价个人

- 既不用于评价个人的能力，也不用于评价个人的绩效。
- 只用于对过程、项目和产品理解、分析、评估、预测和改进工作。
- 在度量活动中将使用特别的步骤来保证分析报告中不被用于评价个人绩效或素质。
- 原因：**为了保持数据的可靠性、客观性和准确性，必须保证度量结果不用于评价数据提供者个人的工作绩效或素质。**

度量活动的角色定义

- 度量工作小组
  - 由专职的度量研究人员和项目协调人员组成。
- 数据提供者
  - 项目中的研发人员，有时还包括服务人员和最终用户
- IT支持者
  - 提供数据收集工具与数据存储设备的人员。

- 根据NASA对度量活动的研究，在度量活动中，各角色的工作量如下：
  - 数据提供者　　2%
  - IT支持者　　3%～10%
  - 度量工作小组　　5%～10%

第2讲　软件项目管理

2.6 风险管理

2.6.1 概述

- 风险就是在项目过程中有可能发生的某些意外事情，而且可能对项目产生巨大的负面影响甚至导致项目失败。
- 风险管理是对项目风险从识别到分析乃至采取应对措施等的一系列过程，它包括将积极因素的影响最大化和使消极因素的影响最小化两方面。

- 风险定义：是指在一定条件下和一定时间内可能发生的各种结果的变化程度。
- 风险的定义大致可分为两类：
  - 广义的风险
    - 在给定的情况和特定时间下，所从事活动的结果有很大的差异性，差异性越大，风险也越大，所面临的损失或收益都可能越大。
  - 狭义的风险
    - 可能消失的东西或者可能受到的伤害。
    - 在从事任何活动时可能面临的损失。

风险的本质

① 不确定性
- 风险可能发生也可能不发生，即不存在概率为100%的风险
- 100%会发生的“风险”实际上是加在项目上的约束。

② 危害性
- 一旦风险变成了现实，就会产生恶性后果或损失。

- 风险的基本要素
  - 风险事件
  - 事件发生的概率
  - 事件的影响
  - 风险的诱因

- 风险的特点
  - 客观性和普遍性
  - 偶然性和必然性。
  - 可变性
  - 多样性和多层次性

风险成本



- 风险的有形成本
  - 直接损失：指人员、经费、设备等的直接流失；
  - 间接损失：直接损失以外的人、财、物、知识等的损失。
- 风险的无形成本
  - 风险的发生减少了项目成功的机会；
  - 风险阻碍了生产率的提高和新技术的应用；
  - 风险会造成资源分配不当，使人们将更多的资源分配投入到风险较小的行业或者项目中。

项目风险的分类

① 技术风险
② 费用风险
③ 进度风险
④ 管理风险
⑤ 社会风险
⑥ 商业风险

按照风险的影响范围分类

(1) 项目风险
(2) 产品风险
(3) 商业风险

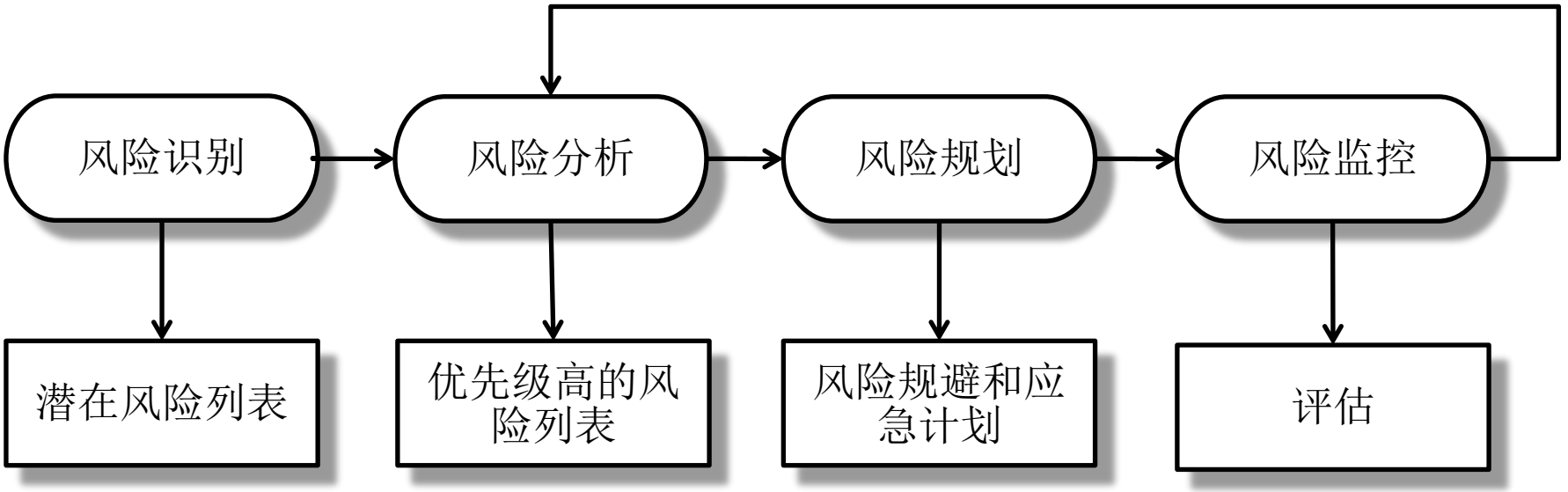
按照风险的可预测性分类

(1) 已知风险
(2) 可预测的风险
(3) 不可预测的风险

项目风险管理

- 项目的风险取决于：
  - 内因：项目本身的特性，主要是项目团队成员对于风险的认识能力及团队成员之间的沟通。
  - 外部：项目所处的外部环境，主要是风险的性质和影响因素的发展变化。
- 按照风险有无预警信息，可将风险管理分为：
  - 无预警信息项目风险管理
    - 采用“救火” 的方法去控制或消减风险后果；
    - 风险转移（通过外包等方式转移风险的方法）
  - 有预警信息项目风险管理
    - 通过收集预警信息去识别和预测风险
    - 通过跟踪风险发生和变化，采取措施控制风险。

风险管理过程



- 风险管理过程贯穿项目全过程。
- 随着有关风险的信息的增多，需要进行重新分析，制定新的风险计划，对风险规避和应急计划要进行修正。

项目风险管理的方法

- 项目风险潜在阶段的管理方法
  - 风险规避
- 项目风险发生阶段的管理方法
  - 风险化解方法
- 项目风险后果阶段的管理方法
  - 削减风险的损失和后果



2.6.2 风险识别

- 识别已知的和可预测的风险。
- 风险识别的主要工作：
  - 识别并确定项目有哪些潜在的风险
  - 识别引起这些风险的主要影响因素
  - 识别项目风险可能引起的后果

- 风险识别方法
  - 风险检查表：利用问卷了解项目和技术方面有哪些风险。
  - 头脑风暴法
  - 情景分析法

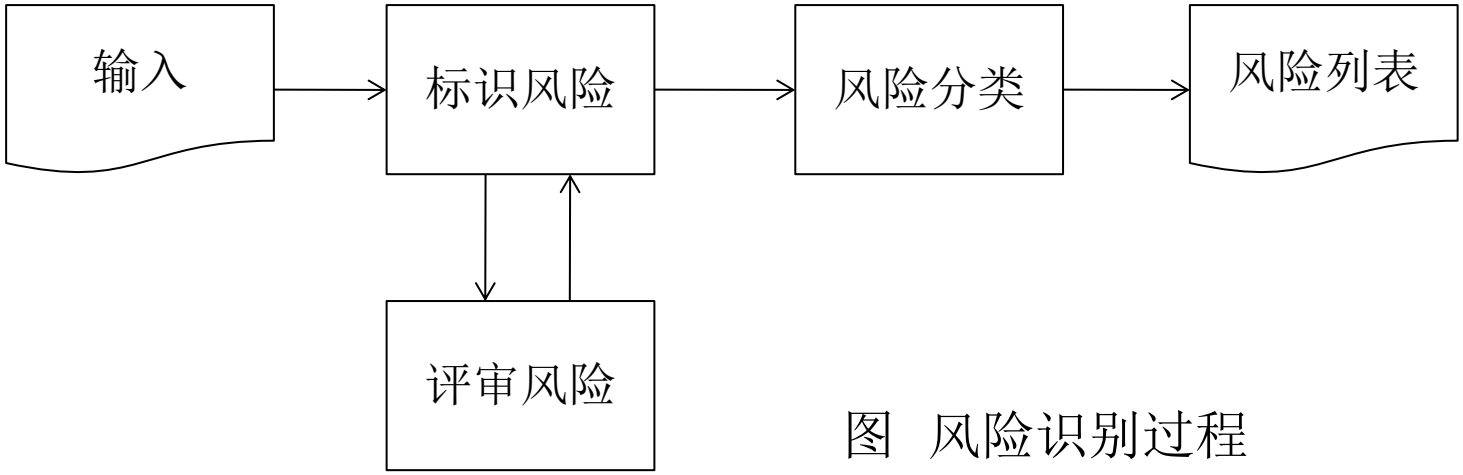


图　风险识别过程

例1：风险及风险类型

风险类型	可能的风险
技术	系统使用的数据库的处理速度不够快 要复用的软件组件有缺陷，限制了项目的功能
人员	招聘不到进符合项目技术要求的工程师 在项目的非常时期，关键性工程师生病，不能发挥作用 工程师所需的培训跟不上
机构	重新进行机构调整，由不同的管理层负责这个项目 开发机构的财务出现问题，必须削减项目预算
工具	CASE工具产生的代码效率低 CASE工具不能被集成
需求	需求产生变化，主体设计要返工 客户不了解需求变更对项目造成的影响
估算	低估了软件开发所需要的时间 低估了缺陷的修补率 低估了软件的规模

例2：针对人力资源风险的Risk Checklist

(1) 可用人员是最优秀的吗？
(2) 按照技能对人员进行了合理组合吗？
(3) 人力足够吗？
(4) 整个项目开发期间人员如何投入？
(5) 有多少人不是全工时投入本项目的工作？
(6) 人们对于手头上的工作是否有正确的目标？
(7) 项目成员是否接受过必要的培训？
(8) 项目的成员是否是稳定的和连续的？

例3：针对技术风险的Risk Checklist

(1) 该技术对你的组织来说是新的吗？
(2) 客户的需求是否需要创建新的算法或I/O技术？
(3) 软件是否需要使用新的或未经证实的硬件接口？
(4) 待开发软件是否要和开发商提供的未经证实的软件接口？
(5) 待开发软件是否要和其功能和性能均未在本领域中得到证实的数据库系统接口？
(6) 产品的需求中是否包括要求采用特定的用户界面？
(7) 产品的需求中是否要求开发某些程序构件，这些构件和你的组织从前开发过的构件完全不同？
(8) 需求中是否要求使用新的分析、设计或测试方法？
(9) 需求中是否要求使用非传统的软件开发方法，如形式化方法，人工神经网络方法？
(10) 需求中对产品性能的约束是否过分严格？
(11) 客户能确定所要求的功能是“可行的”吗？

2.6.3 风险分析预测

- 风险预测又称为风险估算。
- 它试图从两个方面去评价每一个风险：
  - 风险发生的可能性或概率；
  - 如果风险发生了会造成的后果。

- 风险预测活动
  ① 建立一个尺度，以反映风险发生的可能性（尺度可以是布尔值、定性的或定量的）。
  ② 描述风险的后果。
  ③ 估算风险对项目和产品的影响。
  ④ 标注风险整体预测的精确度以免产生误解。

例4：风险分析

风险	出现的可能性	后果
开发机构的财务出现问题，必须削减项目预算	小	灾难性
招聘不到符合项目技术要求的工程师	大	灾难性
在项目的非常时期，关键性工程师生病	中等	严重
要复用软件组件有缺陷，限制了项目的功能	中等	严重
需求发生变化，主体设计要返工	中等	严重
重新进行机构调整，由不同的管理层负责这个项目	大	严重
系统使用的数据库的处理速度不够快	中等	严重
低估了软件开发所需要的时间	大	严重
CASE工具不能被集成	大	可容忍
客户不了解需求变更对项目造成的影响	中等	可容忍
工程师所需的培训跟不上	中等	可容忍
低估了缺陷的修补率	中等	可容忍
低估了软件的规模	大	可容忍
CASE工具产生的代码效率低	中等	可以忽略

风险预测表样本

风险描述	风险类别	发生概率	可能的影响	RMMM
规模估算可能非常低	产品规模	60%	严重	
用户数量大大超过计划	产品规模	30%	轻微	
复用程度低于计划	产品规模	70%	严重	
最终用户抵制该系统	商业风险	40%	轻微	
交付期限紧缩	商业风险	50%	严重	
资金流失	预算风险	40%	灾难	
需求改变	产品规模	80%	严重	
技术达不到预期效果	技术风险	30%	灾难	
缺少对于工具的培训	人力风险	80%	轻微	
人员缺乏经验	人力风险	30%	严重	
人员流动频繁	人力风险	60%	严重	

2.6.4 处理风险的策略

- 所有的风险分析活动都只有一个目的，那就是辅助项目建立处理风险的策略。
- 任何一种有效的策略都必须考虑三个问题：
  ① 风险避免
    - 如果采用主动的策略来处理风险，则避免风险总是最好的策略。
    - 可以通过建立风险缓解计划来达到。
  ② 风险监控
    - 随着项目的进展，风险监控活动也就开始了。
    - 项目管理者监控某些能指出风险概率正在变高还是变低的因素。
  ③ 风险管理及意外事件计划
    - 假设缓解风险的努力失败了，风险变成了现实，就需要风险意外计划。

例：风险处理

- 建立一个风险三元组集合{R_i，L_i，X_i}来评估风险：
  - R_i：风险
  - L_i：风险出现的概率
  - X_i：风险出现会造成的影响
- 人员频繁流动是软件开发组织中的一个普遍存在的风险，建立三元组｛R_0，L_0，X_0｝，其中：
  - R_0 为人员流动风险项；
  - L_0 被估算为0.7，根据历史数据和管理经验；
  - X_0 被预测为对于项目的成本和进度有极严重的影响。

例：人力资源风险避免策略：

- 为了缓解这一风险，项目管理者可以采取如下的策略来降低人员流动：
  (1) 预先与现有人员一同探讨一下人员流动的原因（如：工作条件恶劣、低报酬、激烈的人才竞争等）。
  (2) 在项目开始之前，采取行动来缓解那些被识别出的、且在管理控制之下的原因。
  (3) 一旦项目启动，假设人员会发生流动，并采取一些措施以保证当人员离开时工作的连续性。
  (4) 对项目组进行良好的组织，使得每一个开发活动的信息能够被广泛地传播和交流。
  (5) 定义文档的标准，并建立保证文档能够被及时、正确建立的机制。
  (6) 对所有的工作进行详细的复审，保证不止一个人熟悉该项工作。
  (7) 对每一个关键的技术人员都指定一个后备人员。

例续：风险监控策略

- 随着项目的进展，开始进行风险监控活动。
- 项目管理者监控下列因素：
  (1) 项目组成员对于项目压力的一般态度。
  (2) 项目组的凝聚力。
  (3) 项目组成员彼此之间的关系。
  (4) 与报酬和利益相关的潜在问题。
  (5) 在公司内和公司外工作的可能性。

例续：风险意外计划策略

- 除了监控上述因素之外，项目管理者还应当监控预设的风险缓解步骤的效力。
- 例如：
  - 前述风险避免策略中要求“（5）定义文档的标准，并建立保证文档能够被及时、正确建立的机制”。
  - 如果有关键的人员离开了项目组，就需要启动这个保持工作连续性的机制。
  - 项目管理者应当仔细监控这些文档，以保证每一个文档内容正确，当新员工加入项目组时，能够利用已经建立并得到监控的文档为他们提供必要的信息。

例：风险管理策略

风险	策略
机构的财务问题	拟一份简短的报告，提交高级管理层，说明这个项目将对商业目标有重大贡献
员工招聘问题	告诉客户项目潜在的困难和延迟的可能性，检查要买进的组件
员工生病	重新对团队进行组织，使更多工作有重叠员工可以了解他人的工作
有缺陷的组件	用买进的可靠性稳定的组件更换有潜在缺陷的组件
需求变更	导出可追溯信息来评估需求变更带来的影响，把隐藏在设计中的信息扩大化
机构调整	拟一份简短的报告，提交高级管理层，说明这个项目将对商业目标的重大贡献
数据库的性能	研究一下购买高性能数据库的可能性
低估开发时间	对要买进的组件、程序生成器的效用进行检查

2.6.5 RMMM计划

风险管理计划（RMMM, Risk Mitigation, Monitoring and Management Plan）所有的风险分析工作文档化，并作为整个项目计划中的一部分来使用。

RMMM计划的大纲

1. 引言
   - 1.1 文档的范围和目的
   - 1.2 主要风险综述
   - 1.3 责任划分
     - A. 管理者
     - B. 技术人员
2. 项目风险表
   - 2.1 中止线以上的所有风险的描述
   - 2.2 影响概率及影响的因素
3. 风险缓解、监控和管理
   - 3.1 风险1
     - A. 缓解

     ……
   - 3.n 风险n
     - A. 缓解
       - i. 一般策略
       - ii. 缓解风险的特定步骤
     - B. 监控
       - i. 被监控的因素
       - ii. 监控方法
     - C. 管理
       - i. 意外事件计划
       - ii. 特殊考虑
4. RMMM计划的迭代时间安排表
5. 总结

第2章　软件项目管理

2.7 质量管理

质量的意义

- 在21世纪，这个质量的世纪里，信息产业的质量控制与质量管理将成为质量科学的新增长点，**软件质量是决定人们生活质量的关键**。
- 从20世纪20年代开始，质量科学就伴随着世界经济和科学技术的飞速发展，以及企业管理理论和实践的不断丰富和完善，逐步形成了完整的学科体系。同时，在世界质量科学界出现了以戴明（W. Edwards. Deming）、费根堡姆（A. V. Feigenbaum）、朱兰（J. M. Juran）、石川馨（Kaoru Ishikawa）、克劳斯比（Philiph Crosby）等为代表的世界级质量大师。
- 质量已不再是一种奢侈品，而是任何产品及服务所必须具备的，用户完全满意（TCS-Total Customer Satisfaction）已经成为世界一流企业和跨国公司所共同追求的宏图。

课堂讨论

1987年10月在日本召开的第七届世界质量大会的主题是

“质量第一，永远第一”。

这一口号，得到当时国际社会的公认。

1990年5月在美国旧金山召开的第八届质量大会上，又提出了

“全面质量：90年代国际最紧迫的问题”

这一口号。

请回答下列问题：

你认为这一口号是否适用于现在的软件企业？

传统的质量概念

开发的产品应该符合它的描述。

（Crosby，1979）

Quality is there to meet relevant needs and provide satisfaction to stakeholders.

(Juran, 1999)

理论上，这些定义也应该对所有的产品都适用，但对软件系统而言，存在以下问题：

- 这个描述应该面向客户期望的产品特性，但是开发机构可能有需求（如可维护性）。
- 人们不知道如何用一种明确的方式描述某些质量特性（如可维护性）。
- 写出完备的软件描述非常困难。

2.7.1 软件质量概念

ISO/IEC 9126:1991对软件质量定义是：

软件质量（software quality）是与软件产品满足明确或隐含需求的能力有关的特征和特性的总和。

其含义有4条：

① 能满足给定需要的特性之全体；
② 具有所期望的各种属性的总和的程度；
③ 顾客或用户觉得能满足其综合期望的程度；
④ 软件的组合特性，它确定软件在使用中将满足顾客预期要求的程度。

软件质量特性

特 性	子 特 性	特 性	子 特 性
功能性 Functionality	适合性 **Suitability**	效率 Efficiency	时间特性 **Time behaviour**
	准确性 **Accuracy**		资源特性 **Resource behaviour**
	互操作性 **Interoperability**	可维护性 Maintainability	易分析性 **Analysability**
	依从性 **Compliance**		易更改性 **Changeability**
	安全性 **Security**		稳定性 **Stability**
可靠性 Reliability	成熟性 **Maturity**		易测试性 **Testability**
	容错性 **Fault tolerance**	可移植性 Portability	适应性 **Adaptability**
	易恢复性 **Recoverability**		易安装性 **Installability**
易使用性 Usability	易理解性 **Understandability**		遵循性 **Conformance**
	易学习性 **Learnability**		易替换性 **Replaceability**
	易操作性 **Operability**		

软件质量的6个特性的定义如下：

- **功能性**：与一组功能及其指定的性质的存在有关的一组属性。功能包括满足明确或隐含的需求。
- **可靠性**：与在规定的一段时间和条件下，软件能维持其性能水平能力有关的一组属性。
- **易使用性**：为与使用所需的努力和由一组规定或隐含的用户对这样使用所作的个别评价有关的一组属性。
- **效率**：与在规定的条件下，软件的性能水平与所使用资源量之间关系有关的一组性性。
- **可维护性**：与进行指定的修改所需的努力有关的一组属性。
- **可移植性**：与软件可从某一环境转移到另一环境的能力有关的一组属性。

软件质量6个特性分解为21个子特性：

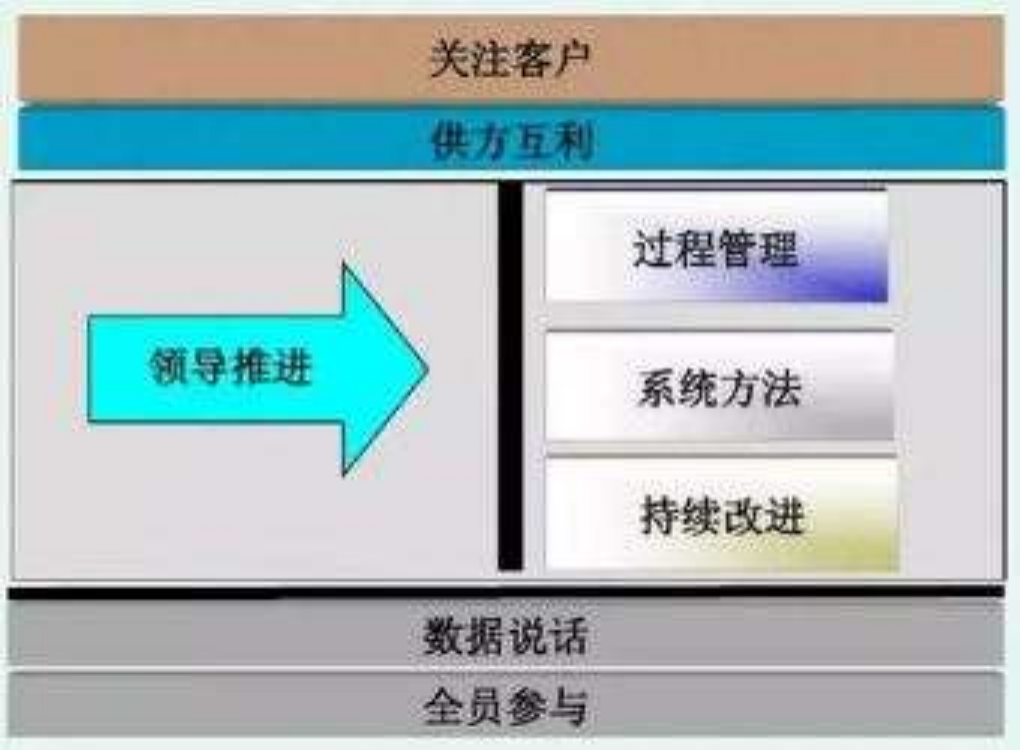
1. **功能性**（5个子特性）
   - 适合性：与规定任何能否提供一组功能及这组功能适合程度有关的软件属性。
   - 准确性：与能否得到正确或相符的结果或效果有关的软件属性。
   - 互操作性：与同其它指定系统进行交互操作的能力有关的软件属性。
   - 依从性：使软件服从有关的标准、约定、法规及类似规定的软件属性。
   - 安全性：与防止对程序及数据的非授权的故意或意外访问的能力有关的软件属性。（保密）
2. **可靠性**（3个子特性）
   - 成熟性：与由软件故障引起失效的频度有关的软件属性。
   - 容错性：与在软件故障或违反指定接口情况下维持规定的性能水平的能力有关的软件属性。
   - 易恢复性：与在失效发生后，重建其性能水平并恢复。
3. **易使用性**（3个子特性）
   - 易理解性：与用户为认识逻辑概念及其适用范围所付出努力有关的软件属性。
   - 易学性：与用户为学习其应用（如运行控制、输入、输出）所付出努力有关的软件属性。
   - 易操作性：与用户为操作和运行控制所付出努力有关的软件属性。

（续）

4. **效率**（2个子特性）
   - 时间特性：与响应和处理时间以及软件执行其功能时的吞吐量有关的软件属性。
   - 资源特性：与在软件执行其功能时，所使用的资源数量及其使用时间有关的软件属性。
5. **可维护性（4个子特性）**
   - 易分析性：与为诊断缺陷或失效原因及为判定待修改的部分所需努力有关的软件属性。
   - 易改变性：与进行修改、排错或适应环境变化所需努力有关的软件属性。
   - 稳定性：与修改造成未预料后果的风险有关的软件属性。
   - 易测试性：与确认经修改软件所需努力有关的软件属性。
6. **可移植性*（4个子特性）***
   - 适应性：与软件无需采用有别于为该软件准备的活动或手段就可能适应不同的规定环境有关的软件属性。
   - 遵循性：使软件遵循与可移植性有关的标准或约定的软件属性。
   - 易替换性：与软件在该软件环境中用来替代指定的其它软件的机会和努力有关的软件属性。
   - 易安装性：与在指定环境下安装软件所需努力有关的软件属性。

质量基本概念

- 从客户的角度看：
  - 质量就是使客户满意
  - 质量就是适用性
  - 质量就是符合客户特定的要求



- 从企业的角度看：
  - 质量一方面反映企业组织的过程能力。如：过程/过程规范的符合度。
  - 质量另一方面反映企业最终产品/服务的交付质量。

软件质量观点：

质量就是软件产品对于某个（或某些）人的价值。

——温伯格（Gerald M. Weinberg ），《质量·软件·管理系统思维》

“正确的软件”及“软件运行正确”



软件质量管理主要活动

- 质量保证：建立起组织的质量体系，这是生产高质量软件的保证。
- 质量规划：为特定的项目编制质量计划。
- 质量控制：定义并设计质量过程，确保软件开发团队严格遵守项目质量规程和标准。

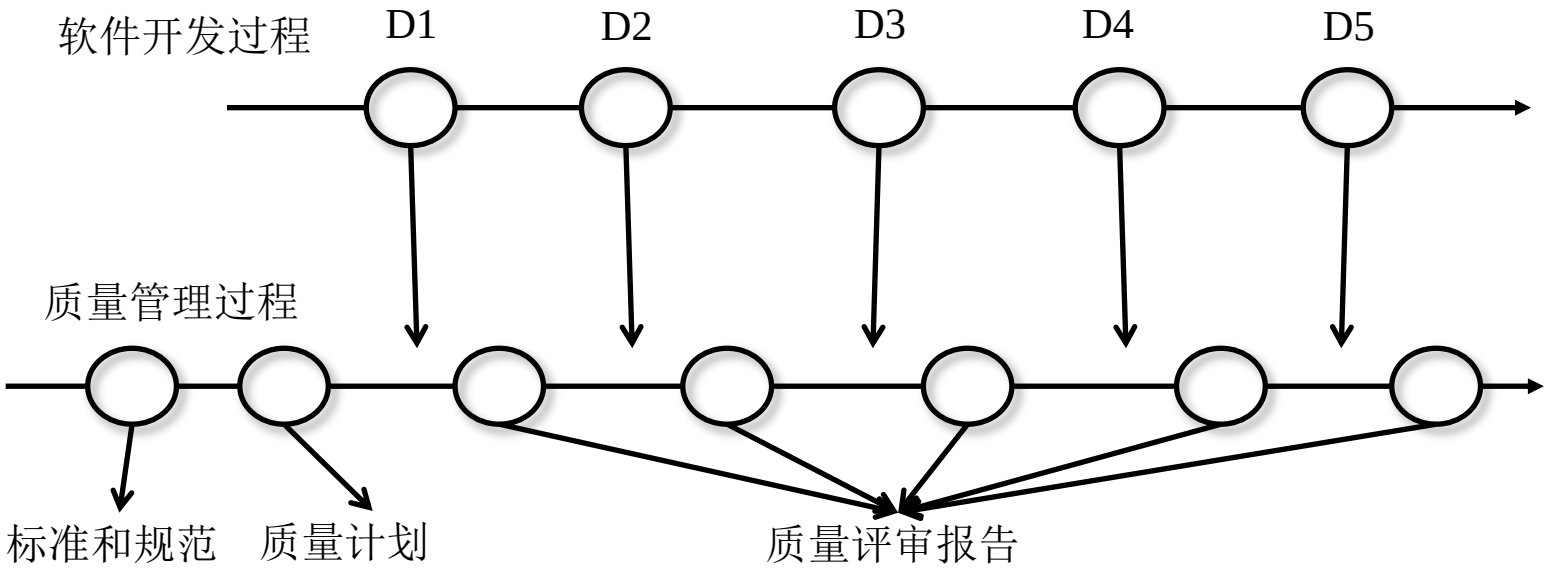


图 质量管理和软件开发

2.7.2 软件质量度量

- 为了在软件开发和维护的过程中，定量地评价软件的质量，必须对软件质量特性进行度量，以测定软件具有要求质量特性的程度。
- 软件质量度量有2类：
  - 预测型
  - 验收型



McCall的质量因素：

- 影响软件质量的因素可以分为两大类：
  ① 可以直接测度的因素（例如，每个功能点的错误）；
  ② 能间接测度的因素（例如，可用性和或维护性）

- McCall提出的影响软件质量的因素的分类。这些软件质量因素集中在产品的三个重要方面：
  ① 它的操作特性
  ② 它承受变化的能力
  ③ 对新环境的适应性

McCall的软件质量因素

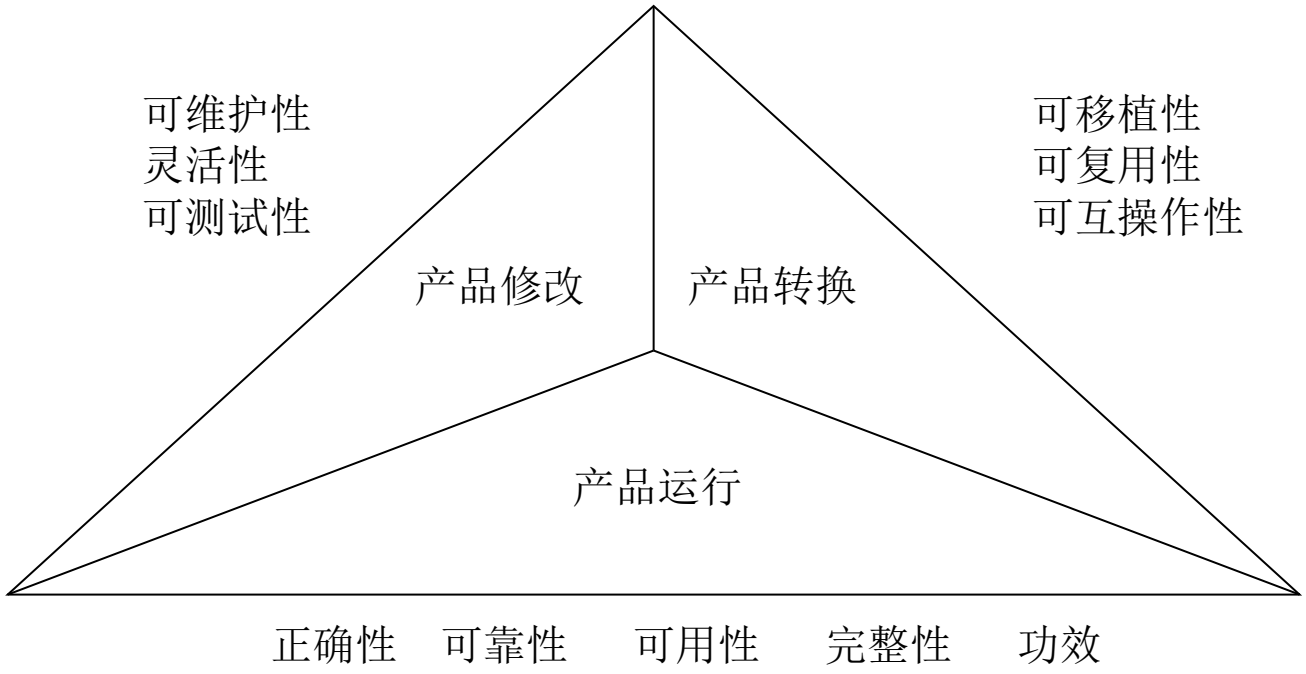


图 McCall的软件质量因素

McCall提供了如下的描述：

- 正确性：一个程序满足它的需求规约和实现用户任务目标的程度。
- 可靠性：一个程序期望以所需的精确度完成 它的预期功能的程度。
- 效率：一个程序完成其功能所需的计算资源和代码的数量。
- 完整性：对未授权人员访问软件或数据的可控制程度。
- 可用性：学习、操作、准备输入和解释程序输出所需的工作量。
- 可维护性：定位和修复程序中一个错误所需的工作量。
- 灵活性：修改一个可运行程序所需工作量 。
- 可测试性：测试一个程序以确保它完成所期望的功能所需的工作量。

（续）

- 可移植性：把一个程序从一个硬件和/或软件系统环境移植到另一个环境所需的工作量。
- 可复用性：一个程序（或一个程序的一部分）可以在另外一个应用程序中复用的程度——这和程序完成的功能的包装或范围相关。
- 互操作性：连接一个系统和另一个系统所需的工作量。
- 很难去开发一个对以上的质量因素的直接测度，因此，定义一组度量，并被用于按照下面的关系为每个因素开发表达式：

$$F_q = c_1 \times m_1 + c_2 \times m_2 + \ldots + c_n \times m_n$$

- 其中：F_q是一个软件质量因素；c_n是回归系数；m_n是影响质量因素的度量值。

McCall评分方案中的度量：

- McCall定义的度量值只能主观地测度，度量可以用检查表形式，来给软件的特定属性进行评分。
- 由McCall提出的评分方案是从0（低）到10（高）的范围。以下是用在评分方案中的度量：
  - 能听度（audibility）：和标准的符合性可被检查的容易程度。
  - 准确度（accuracy）：计算和控制的准确度。
  - 通讯公用度（communication commonality）：标准界面，协议和带宽的使用程度。
  - 完全性（completeness）：所需功能完全实现的程序。
  - 简洁性（conciseness）：以代码行数来评价和谐的简洁程度。
  - 一致性（consistency）：在软件开发项目中一致的设计和文档技术的使用。

（续）

- 数据公用性（data commonality）：在整个程序中对标准数据结构和类型的使用。
- 容错度（error tolerance）：当程序遇到错误时所造成的损失。
- 执行效率（execution efficiency）：一个程序的运行性能。
- 可扩展性（expandability）：结构、数据或过程设计可被扩展的程度。
- 通用性（generality）：程序构件潜在的应用广度。
- 硬件独立性（hardware independence）：软件独立于其运行之上的硬件的程度。
- 检自性（instrumentation）：程序监视它自身的操作并且标识产生的错误的程序。
- 模块性（modularity）：程序部件的功能独立性。

（续）

- 可操作性（operability）：程序操作的容易度。
- 安全性（security）：控制和保护程度和数据的机制的可用度。
- 自包含文档度（self-documentation）：源代码提供有意义的文档程序。
- 简单性（simplicity）：一个程序可以人没有困难的被理解的程度。
- 软件系统独立性（software system independence）：程序独立于非标准编程特性、操作系统特、和其它环境限制的程度。
- 可追溯性（traceability）：从一个设计表示或实际程序部件到 需求说明的能力。
- 可培训性（training）：软件支持使得新用户使用系统的能力。

质量因素和度量

软件质量度量 / 质量因素	正确性	可靠性	功效	完整性	可维护性	灵活性	可测试性	可移植性	可复用性	互操作性	可用性
能听性				×			×				
准确性		×									
通讯公用性										×	
完全性											
复杂性	×										
简洁性		×				×	×				
一致性			×		×	×					
数据公用性	×	×			×	×					
容错度										×	
执行效率		×									
可扩展性			×								
通用性						×		×	×	×	
硬件独立性								×	×		
自检性				×	×		×				
模块性		×			×	×	×	×	×	×	
可操作性			×								×
安全性				×							
自包含文档之稳度					×	×	×	×	×		
简单性		×			×	×	×				
软件系统独立性								×	×		
可追溯性	×										
可培训性											×

2.7.3 软件质量保证

- 软件质量保证(SQA, Software Quality Assurance)是一种应用于整个软件过程的保护性活动。
- SQA包括：
  - 一种质量管理方法
  - 有效的软件工程技术（方法和工具）
  - 在整个软件过程中采用的正式技术复审
  - 一种多层的测试策略
  - 对软件文档及其修改的控制
  - 保证软件遵从软件开发标准的规程
  - 度量和报告机制

百度百科对SQA的说明

软件质量保证（SQA-Software Quality Assurance）是建立一套有计划，有系统的方法，来向管理层保证拟定出的标准、步骤、实践和方法能够正确地被所有项目所采用。

软件质量保证的目的是使软件过程对于管理人员来说是可见的。它通过对软件产品和活动进行评审和审计来验证软件是合乎标准的。

软件质量保证组在项目开始时就一起参与建立计划、标准和过程。这些将使软件项目满足机构方针的要求。

软件质量保证的2个阶段：

- **差异控制是质量管理的核心**。对于每个不同的项目，希望尽可能减小完成项目预计需要的资源和实际使用资源之间的差异，包括人员、设备和时间。
- 软件的质量可以从2种不同的阶段去研究：
  - 设计质量——指在设计过程中为一件产品规定的特征。包括系统的需求、规约和设计方案。
  - 符合质量——指在制造过程中符合设计规格的程序。主要关注实现问题，如果实现符合设计、得到的系统满足系统需求和性能目标，则符合质量较高。

质量保证中角色：

- 软件质量保证由各种任务构成，这些任务分别与两种不同的参与者有关：

  1)软件工程师：通过采用可靠的技术方法和措施、进行正式的技术复审、执行计划周密的软件测试来保证软件质。

  2)SQA小组：负责SQA的计划、监督、记录、分析及报告，辅助软件工程小组（SEG）得到高质量的最终产品。SQA小组是独立与其它工作小组的。

软件质量保证的主要活动：

- 为项目准备SQA计划：该计划在制定项目计划时制定，由所有感兴趣的相关部门复审。该计划将控制由SEG和SQA执行的质量保证活动。
- 参与开发项目的软件过程描述：SEPG为要进行的工作选择一个过程。SQA小组将复审过程说明，以保证该过程与组织政策、内部软件标准、外界所订标准以及软件计划的其它部分相符。
- 复审各项软件活动、对其是否符合定义好的软件过程进行核实：SQA小组识别、记录和跟踪与过程的偏差，并对是否已经改正进行核实。
- 确保软件工作及工作产品中的偏差已被记录在案，并根据预定规程进行处理：偏差可能出现在项目计划、过程描述、采用的标准或技术工作产品中。
- 记录有不符合的部分，并报告给高级管理者：不符合的部分将受到跟踪直至问题得到解决。
- 除上述活动之外，SQA小组还需要协调变化的控制和管理，并帮助收集和分析软件度量信息。

讨论题

1. 项目管理到底要管哪了几方面的内容？
2. 软件生存期与软件项目的生命期有什么区别？
3. 一个程序设计员开发的软件缺陷数低，但是他一直不重视组织的质量标准，这个组织的管理者如何处理这种情况？
4. 你所在单位或项目组进行了哪些度量活动？你认为有需要改进的地方吗？
5. 分析你所在项目存在的风险？